

A PRACTITIONER'S DEEP DIVE

SPARK QUERY OPTIMIZATION AND THE ICEBERG LAKEHOUSE

*Why your queries run the way they do — from Catalyst
to the shuffle to the metadata layer — and how to change it*

Written against: Spark 3.5.2 · Apache Iceberg v2 · Kyuubi
Hive Metastore · Google Cloud Storage · Kubernetes

Alan Phan

2026

Preface

[Section to be written.]

Who This Book Is For

[Section to be written.]

How to Use This Book

[Section to be written.]

Conventions Used in This Book

[Section to be written.]

The four callouts look like this:

Note

Marks a version gotcha, a clarification, or a pointer to a later chapter.

Warning

Marks a default or a pattern that silently costs money, correctness, or data.

Tip

Marks a diagnostic shortcut — a UI panel to check, a one-line command.

Analogy

Marks a plain-language comparison used to build intuition for a mechanism — never a substitute for the mechanism itself, and always pushed only as far as it stays accurate. Skippable if it doesn't click for you; the surrounding prose never depends on it.

Contents

Preface	i
Who This Book Is For	ii
How to Use This Book	iii
Conventions Used in This Book	iv
I How Spark Executes a Query	1
1 The Shape of a Spark Application	2
1.1 A query, end to end	2
1.2 Driver, executors, and the cluster manager	3
1.3 Deploy modes and Spark on Kubernetes	7
1.4 Jobs, stages, and tasks	8
1.5 Narrow versus wide dependencies	10
1.6 Reading the UI for the first time	12
1.7 Summary	13
2 From SQL to Execution: the Catalyst Pipeline	14
2.1 The pipeline, overview	14
2.2 Unresolved to resolved: the catalog	16
2.3 Logical optimization: the rules that matter	17
2.4 Physical planning and strategies	18
2.5 Whole-stage code generation	19

2.6	Reading EXPLAIN	20
2.7	Summary	22
3	How an Executor Uses Memory and CPU	24
3.1	The JVM in one page: bytecode and just-in-time compilation	24
3.2	Heap layout and object representation	25
3.3	The executor memory model	26
3.4	Garbage collection: Parallel, G1, ZGC	29
3.5	Serialization and its cost	30
3.6	Summary	32
II	Making Queries Fast	33
4	The Shuffle	34
4.1	What a partition is: input partitions versus shuffle partitions	34
4.2	The shuffle write path	36
4.3	The shuffle read path	36
4.4	Sizing shuffle partitions	37
4.5	Shuffle compression	40
4.6	Push-based shuffle	40
4.7	Finding and removing unnecessary exchanges	41
4.8	Summary	42
5	Joins	44
5.1	Broadcast hash join	44
5.2	Shuffle hash join	45
5.3	Sort-merge join	46
5.4	Broadcast nested loop join	46
5.5	How the planner chooses	47
5.6	Forcing a strategy	47
5.7	Storage-partitioned joins for bucketed Iceberg tables	48
5.8	Summary	49

6	Adaptive Query Execution	50
6.1	Why runtime beats planning time	50
6.2	Coalescing shuffle partitions	51
6.3	Converting sort-merge to broadcast at runtime	51
6.4	The two broadcast thresholds	51
6.5	Skew-join handling: split and replicate	52
6.6	parallelismFirst and partition count	52
6.7	Reading an AQE plan	53
6.8	Summary	53
7	Partitioning, Bucketing, and Pruning	55
7.1	Partition pruning and the pushdown killers	55
7.2	Dynamic partition pruning	55
7.3	Bucketing to avoid a shuffle	56
7.4	repartition versus coalesce	56
7.5	Column pruning and ReadSchema	56
7.6	Summary	56
8	Caching and Reuse	57
8.1	When caching pays	57
8.2	Lazy cache() versus eager CACHE TABLE	57
8.3	Storage levels and the GC trade-off	57
8.4	How execution and storage memory borrow at runtime	58
8.5	Exchange and subquery reuse	58
8.6	Unpersist and cache thrashing	58
8.7	Summary	58
III	The Iceberg Lakehouse	59
9	Table Formats and the Iceberg Spec	60
9.1	From Hive tables to table formats	60
9.2	The Iceberg spec, format version 2	60

9.3	Snapshots and the metadata tree	60
9.4	Position deletes versus equality deletes	61
9.5	Iceberg, Delta Lake, and Hudi	61
9.6	Summary	61
10	Catalogs and Commits	62
10.1	What a catalog does	62
10.2	The Hive Metastore catalog	62
10.3	The REST catalog	62
10.4	The atomic pointer swap	63
10.5	Optimistic concurrency and retries	63
10.6	Catalog configuration in Spark	63
10.7	Summary	63
11	Iceberg Partitioning and Layout	64
11.1	Hidden partitioning and partition transforms	64
11.2	Partition evolution	64
11.3	Sort orders and Z-ordering	64
11.4	Target file size and the small-files problem	65
11.5	Write distribution modes	65
11.6	Summary	65
12	Reading Iceberg Efficiently	66
12.1	Manifest pruning and metadata filtering	66
12.2	Column statistics and predicate pushdown into Iceberg	66
12.3	Metrics modes: full, truncate, counts, none	66
12.4	Bloom filters	67
12.5	Vectorized reads	67
12.6	Split planning	67
12.7	The metadata tables as a diagnostic	67
12.8	Summary	67

13 Writing and Maintaining Iceberg	68
13.1 Copy-on-write versus merge-on-read	68
13.2 MERGE, UPDATE, and DELETE	68
13.3 Write distribution and skew on write	68
13.4 Compacting data files	69
13.5 Rewriting manifests	69
13.6 Snapshot expiration and orphan files	69
13.7 Metadata management for streaming tables	69
13.8 Summary	69
 IV Operating Spark	 70
14 Tuning Resources and Configuration	71
14.1 Executor sizing from the mechanism	71
14.2 Dynamic allocation	71
14.3 Parallelism defaults	72
14.4 The external and push-based shuffle service	72
14.5 Configuration precedence and profiles	72
14.6 Kyuubi and multi-tenancy	72
14.7 Summary	72
 15 Debugging and Profiling	 73
15.1 The Spark UI as a diagnostic	73
15.2 Event logs and the history server	73
15.3 Reading task, shuffle, spill, and GC metrics	73
15.4 Common failure signatures	74
15.5 async-profiler and flame graphs	75
15.6 Debugging query plans	75
15.7 Summary	75
 16 Streaming into the Lakehouse	 76
16.1 The micro-batch model and the unbounded table	76

16.2	Sources and sinks	76
16.3	Event time and watermarking	76
16.4	Stateful operations and the state store	77
16.5	Output modes and triggers	77
16.6	Exactly-once and the commit	77
16.7	Writing to Iceberg: cadence, small files, maintenance	77
16.8	Summary	77
V	Internals	78
17	Tungsten and Code Generation	79
17.1	The Tungsten binary row format	79
17.2	Off-heap memory management	79
17.3	Cache-aware computation	79
17.4	Whole-stage code generation internals	80
17.5	Expression code generation	80
17.6	Columnar processing and vectorization	80
17.7	Summary	80
18	The Shuffle Framework and the Scheduler	81
18.1	The shuffle manager and shuffle writers	81
18.2	The map output tracker	81
18.3	The DAG scheduler and the task scheduler	82
18.4	RPC and block transfer	82
18.5	Locality and speculation	82
18.6	Summary	82
19	DataSource V2 and Extensions	83
19.1	The DataSource V2 read and write API	83
19.2	Pushdown negotiation	83
19.3	How Iceberg reports splits and statistics	83
19.4	Custom Catalyst rules and SQL extensions	84

19.5	Plugins	84
19.6	Summary	84
A	The Reference Environment and the Running Example	85
B	Configuration Reference	86
C	Reading Query Plans	87
C.1	The six EXPLAIN modes	87
C.2	Reading a physical plan	88
C.3	Scan operators	88
C.4	Exchange operators	89
C.5	Sort operators	90
C.6	Join operators	90
C.7	Aggregation operators	91
C.8	Filter and Project	92
C.9	Whole-stage codegen markers	92
C.10	AQE nodes in a final plan	92
C.11	Predicate-pushdown killers	93
D	Glossary	94
E	Further Reading	96
	References	97

List of Tables

- 1.1 The specimen at a glance, matching Figure 1.3. 10
- 1.2 Narrow versus wide, by operation. `coalesce` is the one surprise: it looks like `repartition` but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.4). 11
- 2.1 The analogy above, mapped exactly onto the pipeline in Figure 2.1. . . 16
- 3.1 The three collectors, at a glance. 30
- 4.1 Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio. 40

List of Figures

1.1	One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.	4
1.2	The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods.	6
1.3	The specimen as one job of three stages. Stage 0 scans and filters trips; stage 1 scans <code>dim_route</code> ; both write shuffle files partitioned by <code>route_id</code> . Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.	9
1.4	A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.	10
2.1	The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.	15

3.1	An executor's memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager's container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.	28
3.2	A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.	31
4.1	The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file's location with the driver's map output tracker, and the reduce task asks the tracker where to fetch from. . . .	37
4.2	For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band. . .	38
5.1	The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality. . . .	45
6.1	Without AQE, one oversized partition runs as a single task long after the rest of the stage has finished. AQE splits it into sub-partitions and replicates the small side across them, turning one straggler into several parallel, evenly-sized tasks.	53

Part I

How Spark Executes a Query

Chapter 1

The Shape of a Spark Application

A query you would actually write — a week of trips joined to a route dimension, aggregated to an on-time rate — runs in four minutes. This chapter does not make it faster. It uses it as a specimen: you run it, open the Spark UI, and learn to name every moving part — the driver, the executors, the one job, its stages, the shuffle between them, and the one task doing more than the rest. By the end you can read a running job instead of watching a progress bar.

1.1 A query, end to end

Here is the specimen. It asks, for each bus route, what fraction of trips in the first week of March arrived on time. It runs against the `trips` table and the `dim_route` dimension introduced in Appendix A; `trips` is an Iceberg table partitioned by `day(event_ts)`, holding a few hundred million rows for the month.

```
1 SELECT r.route_name ,
2         AVG(CASE WHEN t.arrival_delay_s <= 120 THEN 1.0 ELSE 0.0
3             END) AS on_time_rate
4 FROM trips t
5 JOIN dim_route r ON t.route_id = r.route_id
6 WHERE t.event_ts >= TIMESTAMP '2026-03-01 00:00:00'
7        AND t.event_ts <  TIMESTAMP '2026-03-08 00:00:00'
```

```
7 GROUP BY r.route_name  
8 ORDER BY on_time_rate;
```

Listing 1.1: The specimen query

Submit it and Spark does not simply “run the SQL.” The text is parsed into a tree, resolved against the catalog, rewritten by the optimizer, turned into a plan of physical operators, and compiled to Java bytecode. That whole compilation is Chapter 2’s subject; for now, take it as given that by the time anything executes, Spark holds a *physical plan*: a tree of operators such as *scan*, *filter*, *exchange* (a shuffle), *sort-merge join*, and *hash aggregate*.

Executing that plan is a distributed affair. The plan is split into a *job*, the job into *stages*, and each stage into *tasks* that run in parallel on *executors*. The result flows back to the process that submitted the query, the *driver*. On the reference cluster this specimen produces one job, three stages, and a few hundred tasks, and it takes about four minutes.

Why four minutes and not one, or forty? That question is what the rest of the book answers. This chapter’s job is narrower: to give you the vocabulary and the tools to *see* the structure of the run, so that later chapters have something concrete to change. Everything that follows — joins, the shuffle, adaptive execution, the Iceberg metadata layer — is a way of moving less data, moving it less often, or moving it more evenly. You cannot decide to move data differently until you can see where it moves now.

1.2 Driver, executors, and the cluster manager

A Spark *application* is one driver plus a set of executors, all working on your queries until the application exits (Figure 1.1).

The **driver** is the process that runs your program. It holds the `SparkSession`, compiles each query into a plan, breaks the plan into stages and tasks, hands tasks to executors, tracks their progress, and gathers results. It is a single process and a single point of failure: if the driver dies, the application dies. It is also where a `collect()` or a `toPandas()` lands every row it pulls back, which is the most common way to run the driver out of memory (Chapter 3, Section 15.3).

An **executor** is a JVM process that does the actual work. Each executor has a fixed

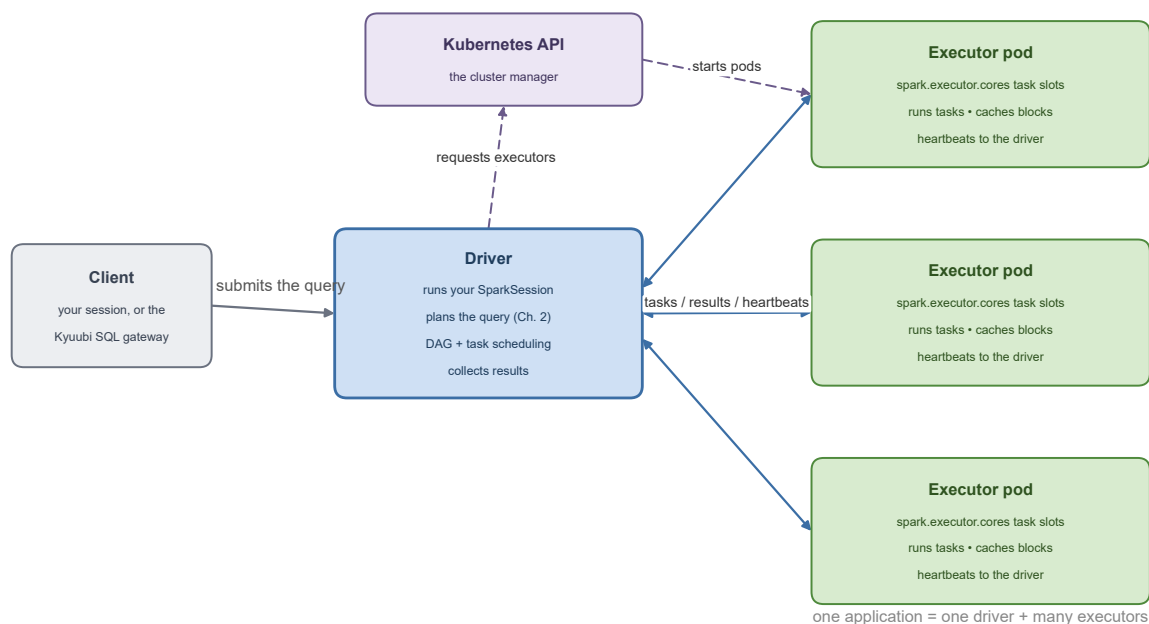


Figure 1.1: One application: a single driver process that plans and coordinates, and a set of executor processes that run tasks and cache data. The cluster manager — here the Kubernetes API — hands out the containers the processes run in.

number of *task slots* — one per core, set by `spark.executor.cores` — and runs one task per slot at a time. Executors also hold cached blocks and the map-side output of shuffles, and they send regular heartbeats to the driver so it knows they are alive. An application typically has many executors and exactly one driver.

The **cluster manager** owns the machines and hands out the containers that the driver and executors run in. Spark supports several: its own standalone manager, Hadoop YARN, and Kubernetes (Apache Mesos support was removed in Spark 3.2). The reference stack uses Kubernetes, so “container” means “pod” and the cluster manager is the Kubernetes API server.

Note

The driver–executor split is why two very different failures look different. An executor lost to an out-of-memory kill takes its tasks down, and Spark simply re-runs them elsewhere; the job slows but survives. The driver running out of memory — usually from pulling too much data back — kills the whole application at once, with no retry.

Analogy

A busy restaurant kitchen has the same three roles. The **driver** is the head chef at the pass: reading each ticket, deciding how the dish comes together, calling out instructions — and never touching a pan. The **executors** are the line cooks, each at their own station with their own pans (task slots), doing the actual cutting and cooking and calling back to the pass when a plate is ready. The **cluster manager** is the staffing agency: when the chef needs more hands on a busy night, it sends cooks over, but it never runs a station itself. Push it further than that and it breaks — no kitchen survives a chef who cannot cook a single plate — but the division of labour is exactly right: one planner, many workers, and an outside party that only ever supplies the workers.

Following the specimen from submission to result

Putting the three roles together, here is what happens end to end when you run the query from Listing 1.1, step by step (Figure 1.2).

1. **Submit.** Your SQL client — here a connection to the Kyuubi gateway — sends the query *text* to the driver. Nothing is parsed on your side.
2. **Compile (driver).** The driver parses the text, resolves `trips` and `dim_route` against the catalog, runs the optimizer, and produces a physical plan (Chapter 2). Its DAG scheduler then cuts that plan into three stages at the shuffle and builds a set of tasks for each — one task per input partition.
3. **Acquire executors (cluster manager).** If the driver does not already have enough executors, it asks Kubernetes for more pods. Under Kyuubi the pool is usually already warm, so this step does nothing and the query starts at once.
4. **Run the map stages (executors).** The driver sends stage 0 and stage 1 tasks to free task slots, each task carrying its plan fragment and the id of the one partition it must process. Stage 0's roughly 180 tasks each read one file split of the March `trips` partitions from GCS, apply the `event_ts` filter, and write small files to the executor's *local* disk, bucketed by `route_id`. Stage 1 does the same for `dim_route`.

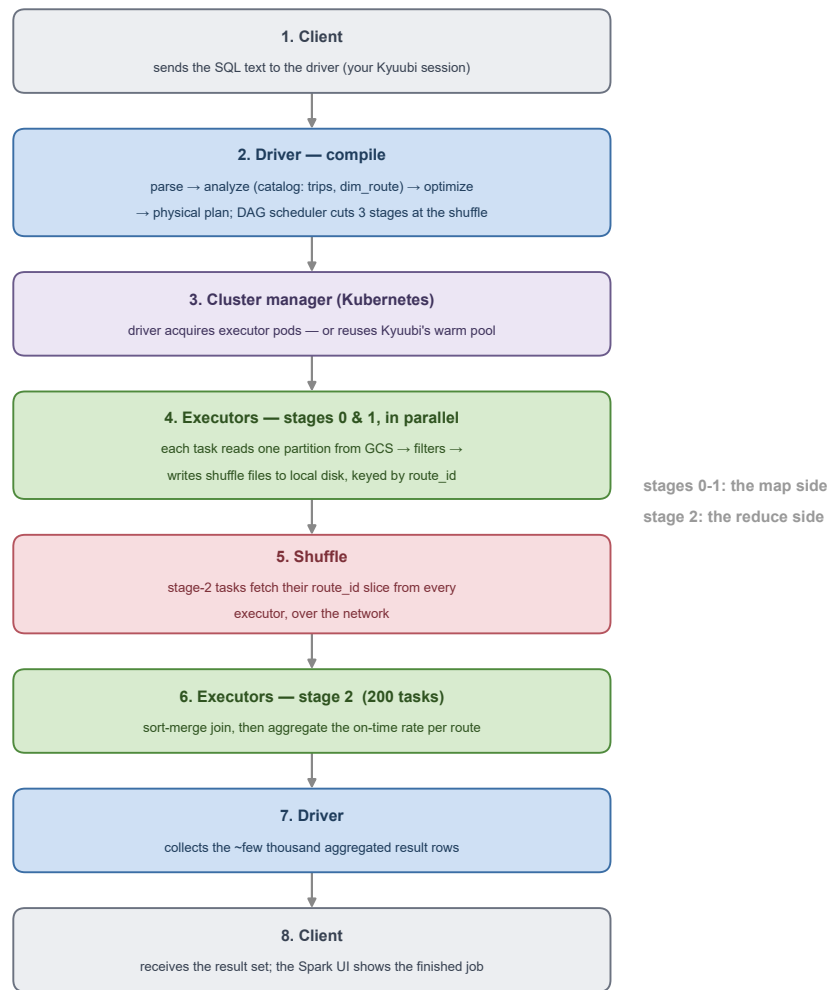


Figure 1.2: The specimen query from submission to result. The driver compiles and schedules but never touches the data; executors do all the reading, shuffling, and computing; the cluster manager only supplies the pods.

The two stages run concurrently, and every task reports its metrics to the driver as it finishes.

5. **Shuffle.** Once both map stages are done the driver launches stage 2 — 200 tasks, one per value of `spark.sql.shuffle.partitions`. Each stage-2 task connects to *every* executor that ran a map task and fetches the shuffle files for its slice of `route_id`. This all-to-all transfer over the network is the shuffle, and it is why the map side (stages 0–1) and the reduce side (stage 2) are separate stages.
6. **Run the reduce stage (executors).** Each stage-2 task now holds every row for its routes, from both tables. It sorts and merges them, performs the join, and aggregates the on-time rate per route.
7. **Collect (driver).** The result is one row per route — a few thousand rows — so the driver gathers them from the stage-2 tasks. A query returning millions of rows would overwhelm the driver at exactly this step (Chapter 3).
8. **Return.** The driver sends the result set to the client and marks the job complete. The Spark UI now shows the finished job, its three stages, and every task’s metrics — the view you learn to read in Section 1.6.

Throughout, the driver is also doing bookkeeping the diagram leaves out: tracking which tasks have finished, re-sending any that failed to a different executor, and updating the UI. It never reads or writes the data itself — only the plan, the schedule, and the final small result.

1.3 Deploy modes and Spark on Kubernetes

Where the *driver* runs is the *deploy mode*. In **client mode** the driver runs in whatever process launched the application — your shell, a notebook kernel, or a long-lived gateway. In **cluster mode** the launcher asks the cluster manager to start the driver as just another container, then disconnects. Client mode is convenient for interactive work because the driver is local to you; cluster mode is standard for scheduled jobs because nothing on your side has to stay running.

On Kubernetes the sequence for a cluster-mode job is: `spark-submit` calls the Kubernetes API to create a *driver pod*; the driver pod starts, and its Spark code calls the same API to create *executor pods*; each executor pod starts, connects back to the driver, and registers its task slots. From then on the driver schedules tasks onto executors directly, and the Kubernetes API is only involved again when pods are added or removed — for example under dynamic allocation (Section 14.1).

The reference stack does not usually run `spark-submit` per query. **Kyuubi** runs a long-lived driver (a persistent `SparkSession`, one per user or group) and you send SQL to it over a connection, the way you would to any database. The engine, stages, and tasks are all still there; Kyuubi just keeps the driver warm so you are not paying startup cost on every query and so many users can share a pool of executors.

1.4 Jobs, stages, and tasks

Spark transformations are *lazy*: `filter`, `join`, `groupBy`, and the rest build up a plan but run nothing. Work starts only when an *action* asks for a result — a write, a count, a collect, or the implicit collect that returns rows to a SQL client. Each action triggers one **job**.

Within a job, the driver's *DAG scheduler* cuts the plan into **stages**. The cut always falls at a shuffle: a stage is the longest run of work that can be pipelined without moving data between executors. Everything up to a shuffle goes in one stage; the shuffle ends it; what comes after starts a new one. A stage that reads a shuffle cannot begin until the stage that writes it has finished, which is why stage boundaries are also the points where a job can stall.

A stage is executed as a set of **tasks** — one per partition of the stage's input, each doing identical work on its own slice of the data. A task is the unit Spark schedules onto an executor's task slot; one task occupies one core for its duration. The number of tasks in a stage is therefore the number of partitions, and it sets how much parallelism the stage can use: 8 tasks on a cluster with 200 free slots leaves 192 idle.

Figure 1.3 shows the specimen's shape. Because `dim_route` has no statistics that would let the planner broadcast it (an Iceberg table usually does not, until you `ANALYZE`

it — Chapter 5), the planner chooses a sort-merge join: both inputs are shuffled so that rows with the same `route_id` land on the same executor. That gives:

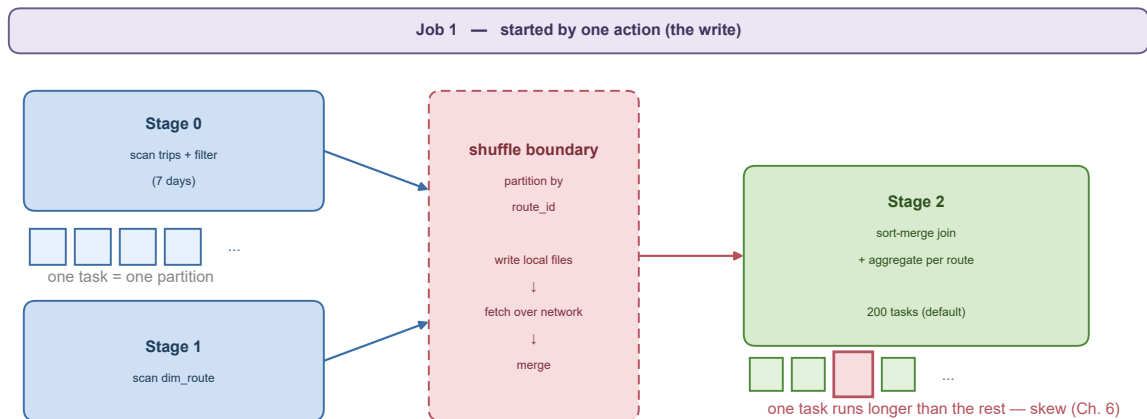


Figure 1.3: The specimen as one job of three stages. Stage 0 scans and filters trips; stage 1 scans `dim_route`; both write shuffle files partitioned by `route_id`. Stage 2 reads from both, joins, and aggregates. Each stage runs one task per input partition. One task in stage 2 runs far longer than its peers — the first sighting of skew.

- **Stage 0** — scan the March partitions of `trips`, apply the timestamp filter, and write shuffle files keyed by `route_id`. Roughly 180 tasks, one per input file split.
- **Stage 1** — scan `dim_route` and write shuffle files keyed by `route_id`. A handful of tasks; the table is tiny.
- **Stage 2** — fetch both sides' shuffle files, merge them on `route_id`, join, and aggregate per route. 200 tasks, because `spark.sql.shuffle.partitions` defaults to 200 (Chapter 4). The final `ORDER BY` is small enough to finish inside this stage.

Stage	Reads	Tasks	Writes / does
0	trips, March partitions, filtered	~180	shuffle files, keyed by route_id
1	dim_route	a handful	shuffle files, keyed by route_id
2	stage 0 + stage 1 shuffle output	200	sort-merge join, aggregate, sort

Table 1.1: The specimen at a glance, matching Figure 1.3.

The ORDER BY on the aggregated result is cheap — a few thousand rows — so it does not add a stage here. On a larger result it would.

1.5 Narrow versus wide dependencies

The reason a shuffle ends a stage is the kind of dependency it creates between partitions (Figure 1.4).

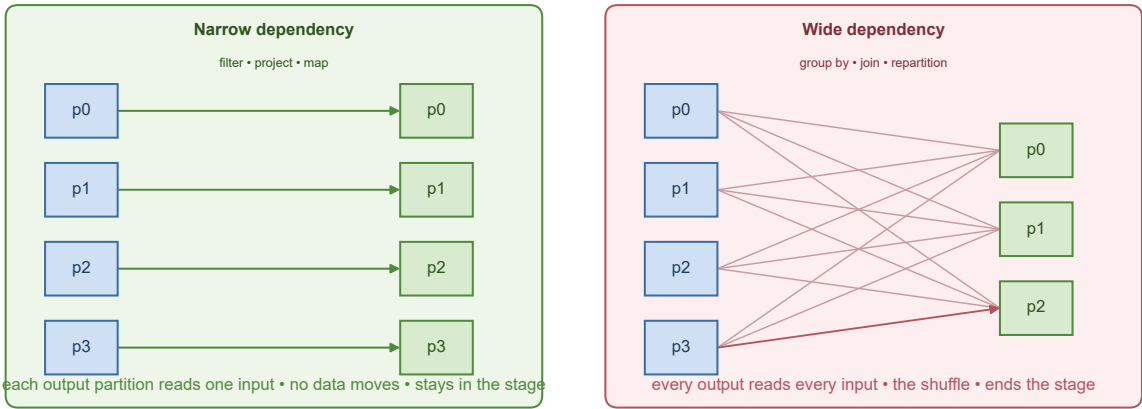


Figure 1.4: A narrow dependency: each output partition is built from one input partition, so the work pipelines in place. A wide dependency: each output partition draws from every input partition, so the data must be redistributed — a shuffle — and the stage ends.

A **narrow** dependency is one where each output partition depends on a bounded,

known set of input partitions — usually exactly one. `filter`, `select`, and `map` are narrow: partition 3 of the output is partition 3 of the input with a function applied. Narrow operations chain together with no data movement, which is why an entire run of them collapses into a single stage and executes as one pipelined pass per partition.

A **wide** dependency is one where an output partition can depend on *every* input partition. `groupBy`, `join` (unless one side is broadcast), `distinct`, `repartition`, and window functions are wide: to put all rows for `route_id = 47` in one place, Spark must look at every input partition, because any of them might hold such a row. That redistribution is the shuffle, and it is the wall between stages.

Operation	Dependency	Why
<code>filter</code> , <code>select</code> , <code>withColumn</code>	narrow	one input partition maps to one output partition
<code>union</code>	narrow	partitions are concatenated, no keys compared
<code>coalesce(n)</code> (fewer partitions)	narrow	merges adjacent partitions without moving keys
<code>groupBy</code> , <code>join</code> (no broadcast), window functions	wide	equal keys from anywhere must co-locate
<code>distinct</code>	wide	compiles to an aggregation
<code>repartition(n)</code> , <code>orderBy</code> (global)	wide	explicit or ordering-driven redistribution

Table 1.2: Narrow versus wide, by operation. `coalesce` is the one surprise: it looks like `repartition` but avoids a shuffle by only ever merging partitions, never splitting or reshuffling keys (Section 7.4).

This is the single most useful idea in the book: **cost lives at stage boundaries**. Narrow work is cheap and scales cleanly. Wide work writes data to disk, moves it over the network, and re-reads it, and it forces the whole cluster to wait at a barrier. Almost every technique in Chapter 4 through Chapter 8 is about having fewer wide dependencies, making the ones you keep move less data, or making them move it more evenly.

1.6 Reading the UI for the first time

The Spark UI — served by the driver, on port 4040 by default, and preserved after the fact by the history server (Section 15.3) — has a tab for each level of the model.

SQL / DataFrame The query as a plan tree, annotated with the number of rows and the time spent at each operator after execution. This is where you confirm which join strategy ran and count the *Exchange* nodes; each Exchange is one shuffle.

Jobs One row per action. The specimen shows a single job.

Stages The stages of each job, with a timeline and, for every stage, a *Summary Metrics* table: the minimum, 25th percentile, median, 75th percentile, and maximum across the stage’s tasks for duration, shuffle read size, records, spill, and GC time.

Executors One row per executor: task slots, memory used, disk spilled, and total GC time. A cluster where one executor shows far more GC than the others is usually holding skewed data.

Storage Cached datasets and how much of each is actually resident.

For the specimen, open the Stages tab and look at stage 2’s Summary Metrics. The median task duration is a few seconds; the maximum is close to a minute. One task is doing a large multiple of the work of the typical task. That is *data skew*: one `route_id` — a busy trunk route — holds far more trips than the others, so the task that got that route’s partition runs alone while the rest of the stage has finished. You cannot fix it yet. Chapter 6 and Chapter 5 will. But you can now *see* it, and seeing it is the prerequisite for everything else.

Tip

Four numbers tell you most of what is wrong with a slow stage: max versus median *task duration* (skew), *shuffle read/write* size (how much data the wall is moving), *spill* (not enough execution memory, Chapter 3), and *GC time* (too much, Section 3.4). Check them in that order.

1.7 Summary

You can now describe what happens when a query runs. The *driver* plans it and coordinates; *executors* run *tasks*, one per core, on partitions of the data; the *cluster manager* (Kubernetes here) provides the containers. An action triggers a *job*; the job is cut into *stages* at every shuffle; a stage runs one task per partition. A *narrow* dependency keeps work in place; a *wide* dependency forces a shuffle and ends the stage, and that is where a query spends most of its time. In the Spark UI you can find the job, its stages, and their tasks, identify a stage boundary as a shuffle from the Exchange nodes in the SQL tab, and spot an uneven stage from the gap between its maximum and median task time.

The specimen still takes four minutes. Chapter 2 opens up how Spark decided on this plan; Chapter 4 onward is how you change it.

Exercises

1. Run two queries against `trips`: `SELECT count(*) FROM trips` and `SELECT route_id, count(*) FROM trips GROUP BY route_id`. Open the Stages tab for each. Which produced a second stage, and why?
2. In the SQL tab for the specimen query, count the *Exchange* nodes. Match each one to a stage boundary in Figure 1.3.
3. In stage 2's Summary Metrics, record the median and maximum task duration and the maximum shuffle-read size. Which of the four numbers from the tip is the outlier?

Chapter 2

From SQL to Execution: the Catalyst Pipeline

A SQL string does not run. It is parsed into a tree, resolved against the catalog, rewritten by rules, turned into a physical plan by choosing strategies, and compiled to JVM bytecode. Every change you can make to a query's performance is a change to one of those stages. This chapter traces the specimen query from Chapter 1 through all of them and teaches you to read `EXPLAIN FORMATTED` well enough to predict the physical plan before you run it.

2.1 The pipeline, overview

Catalyst is the optimizer inside Spark SQL. It is a library for building and rewriting trees, plus a fixed sequence of passes that turn a query into an executable plan (Figure 2.1). Every DataFrame operation and every SQL statement goes through the same pipeline; the DataFrame API is just a second front end that builds the same tree the SQL parser does.

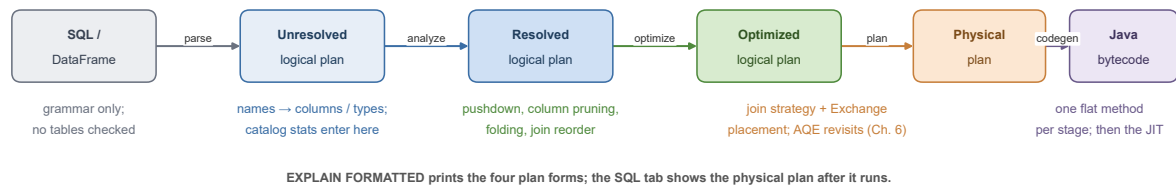


Figure 2.1: The compilation pipeline. Parsing produces a tree that is pure syntax. Analysis resolves names against the catalog. Logical optimization rewrites the tree with rules. Physical planning picks concrete operators and shuffle points. Code generation emits Java that the JIT compiles. The side notes name what each stage decides.

Analogy

Recall the kitchen from Section 1.2. The order goes through the same five stages there as a query does here. A waiter writes the spoken order onto a kitchen ticket in shorthand, without checking whether the kitchen actually has those ingredients tonight — that is *parsing*: pure syntax. The expediter then checks the ticket against the pantry and the menu, catching an order for a dish that is not served or an ingredient that is out — that is *analysis*, against the catalog. The head chef mentally simplifies the prep — “we’re already chopping onions for table 4, chop extra for this ticket too” — without changing what gets served: that is *logical optimization*. Stations and techniques are then assigned — grill or oven, this cook or that one — based on what is actually fastest tonight: that is *physical planning*. And a dish the kitchen has made a thousand times gets cooked by hands that no longer think through each step: that is *whole-stage code generation* (Section 2.5).

In the kitchen	In Catalyst	What changes
waiter writes the ticket	parse	text → syntax tree; nothing checked
expediter checks the pantry & menu	analyze	names resolved against the catalog
chef simplifies the prep	logical optimization	rule-based rewrites (Section 2.3)
stations & techniques assigned	physical planning	operators & shuffle points chosen
practiced hands, no thinking	whole-stage codegen	generated Java, then the JIT

Table 2.1: The analogy above, mapped exactly onto the pipeline in Figure 2.1.

Four representations matter, and you can print all of them (Section 2.6):

1. the **unresolved logical plan** — the parse tree; relations and columns are just names.
2. the **resolved (analyzed) logical plan** — names bound to real tables and typed columns.
3. the **optimized logical plan** — the same query, rewritten to do less work.
4. the **physical plan** — a tree of executable operators, with the shuffles made explicit as *Exchange* nodes.

The physical plan is what Chapter 1 called “the plan.” Reading it, and knowing which pass produced each part of it, is what lets you look at a query and say in advance how many stages it will have and where they will stall.

2.2 Unresolved to resolved: the catalog

The parser only checks grammar. `SELECT r.route_name FROM trips t JOIN dim_route r ...` parses whether or not those tables exist; the result is a tree whose relation and attribute nodes are still marked *unresolved* — `trips` and `r.route_name` are held as names, not yet bound to anything.

The *analyzer* resolves them. It asks the **catalog** what `trips` and `dim_route` are, gets back their schemas, and replaces each unresolved node with a concrete one: `route_name`

becomes a `string` column belonging to `dim_route`, `arrival_delay_s` becomes an `int` on `trips`, the `JOIN` condition is type-checked, and `AVG(...)` is bound to the aggregate function. If a name does not resolve, this is where the query fails — before any work is scheduled.

In the reference stack the catalog is a Spark *catalog plugin* pointed at an Iceberg REST endpoint (backed by the Hive Metastore); Chapter 10 covers what it stores and how commits move through it. What matters here is that the catalog is also where *table statistics* come from — row counts, column value ranges, distinct counts. Spark’s cost-based optimizer uses them to order joins and to decide what is small enough to broadcast (Chapter 5). An Iceberg table exposes a total file size and row count from its metadata, but not per-column statistics unless you have run `ANALYZE TABLE ...COMPUTE STATISTICS`. This is why so many lakehouse joins pick the wrong strategy at planning time and have to be rescued by a hint or by AQE.

2.3 Logical optimization: the rules that matter

The optimizer applies *rules* — functions from a logical plan to a better logical plan — grouped into *batches* that run repeatedly until the plan stops changing. Spark ships around a hundred rules. A handful account for most of the speed-up, and all of them do the same thing: move work earlier, or delete it.

Predicate pushdown. A filter is pushed as close to the scan as the semantics allow, so fewer rows enter the rest of the plan. In the specimen the `event_ts` range moves into the `trips` scan, where it becomes a *partition filter* (Iceberg skips whole days) and a *pushed filter* (Parquet skips row groups). Chapter 7 is about keeping this rule working; a function or a `CAST` on the filtered column defeats it.

Column pruning. The scan reads only the columns the query references. The specimen touches `route_id`, `arrival_delay_s`, and `event_ts` from `trips` and `route_id`, `route_name` from `dim_route` — so a fifty-column `trips` table is read five columns wide. A `SELECT *` anywhere in a subquery turns this off.

Constant folding and simplification. Expressions with no runtime input are evaluated once at planning time; the `TIMESTAMP '2026-03-01 ...'` literals are computed here, not per row. Boolean and null simplifications (`x AND true`, `x = x`, `IS NOT NULL` on a non-nullable column) are folded away.

Join reordering. With reliable statistics and cost-based optimization enabled (`spark.sql.cbo.enabled` and `spark.sql.cbo.joinReorder.enabled`), a multi-way join is reordered to keep intermediate results small — join the pair that produces the fewest rows first. Without statistics Spark keeps the written order, which is one more reason Iceberg tables benefit from `ANALYZE`.

Note

Most optimizer rules are *heuristic*: they rewrite the plan into a form that is almost always better, using no cost model. Only join selection and join reordering are cost-based, and only when statistics exist. When people say “the optimizer is smart,” they usually mean the heuristic rules, which are reliable; the cost-based part is only as good as the catalog’s numbers.

2.4 Physical planning and strategies

The optimized *logical* plan still says only *what* to compute. The planner turns each logical operator into one or more *physical* operators by applying *strategies*:

- a logical `Join` becomes `BroadcastHashJoin`, `ShuffledHashJoin`, or `SortMergeJoin`, chosen from the estimated sizes of its inputs and `spark.sql.autoBroadcastJoinThreshold` (Chapter 5).
- a logical `Aggregate` becomes a partial `HashAggregate` on each partition, an `Exchange` to bring equal keys together, and a final `HashAggregate`.
- a logical `Sort` that needs a global order becomes a range-partitioning `Exchange` followed by a local `Sort`.

The planner also inserts `Exchange` (shuffle) and `Sort` nodes wherever an operator’s required input distribution is not already satisfied — and, crucially, *removes* them where

it is. If two joins share a key and the data is already partitioned by it, the second join reuses the first's exchange. This is why the *number of Exchange nodes in the physical plan* is the single best predictor of a query's cost, and why Chapter 1 told you to count them in the UI.

For the specimen, with no broadcastable statistics on `dim_route`, the planner produces: two scans, each followed by an `Exchange hashpartitioning(route_id, 200)`, a `SortMergeJoin`, a partial and final `HashAggregate` (which need no new exchange, because the join already partitioned by `route_id`), and a final small sort. Three stages, exactly as Figure 1.3 showed.

2.5 Whole-stage code generation

A physical plan could be executed by walking the operator tree and calling `next()` on each node once per row — the “Volcano” iterator model. For a chain of `Filter` → `Project` → `HashAggregate`, every row passes through three virtual method calls, and the boxing and the short-lived row objects between operators usually cost more than the actual filtering and arithmetic.

Spark does not do this. For a chain of operators within a stage it *generates Java source* for the whole chain — one method that scans, filters, projects, and feeds the aggregate in a tight loop, with no virtual call, no intermediate row object, and field access straight out of the packed binary row (Chapter 17). That method is compiled to bytecode and then by the JVM's JIT to native code.

In a printed plan you see this as a numbered marker. Operators fused into one generated function are wrapped: `*(2) HashAggregate,*(2) Project,*(2) Filter`. The `*(n)` means “part of generated stage n”; the number is just a sequential id, not an execution order. An operator *without* a star is a boundary where fusion stopped and Spark falls back to row-at-a-time.

A stage stops being generated for a handful of reasons, and a plan missing stars where you expect them is usually hitting one:

- an `Exchange` or `BroadcastExchange` — you cannot fuse operators across a network boundary, so codegen always restarts on the far side of a shuffle;

- a stage whose input or output schema has more columns than `spark.sql.codegen.maxFields` (100 by default);
- a generated method that would exceed the JVM’s 64 KB bytecode limit for a single method (`spark.sql.codegen.hugeMethodLimit`); Spark logs `Found too long generated codes` and drops that stage to the interpreter;
- a compile failure in the generated Java, after which Spark silently falls back (`spark.sql.codegen.fallback`) — the only trace is a `WARN` in the executor log;
- an imperative (non-declarative) aggregate or UDAF, which some complex user-defined aggregates are.

`EXPLAIN CODEGEN` prints the generated Java for every whole-stage stage, which is how you confirm what actually got fused.

2.6 Reading EXPLAIN

`EXPLAIN` prints the plans without running the query. There are six forms, selected by a keyword in SQL or a mode argument to `df.explain(...)`.

Mode	Shows
(default)	the physical plan only — the quick check for join strategy, shuffle count, codegen
EXTENDED	all four plans: parsed, analyzed, optimized logical, and physical — use to see what a rule did
FORMATTED	the physical plan as a compact tree <i>plus</i> a numbered list expanding each node’s detail — the one to learn
COST	the optimized logical plan annotated with the planner’s row-count and size <i>estimates</i> — use to see why it did not broadcast
CODEGEN	the physical plan plus the generated Java for each whole-stage stage

`EXPLAIN FORMATTED` has two parts: a compact tree, then a numbered list where each node’s details are spelled out. Read the tree **bottom-up** — the leaves are scans, the root is the final operator.

```

== Physical Plan ==
*(3) Sort (13)
+- *(3) HashAggregate (12)                -- final aggregate
   +- *(2) HashAggregate (10)             -- partial aggregate
      +- *(2) SortMergeJoin Inner (9) -- both sides shuffled; not broadcast
         :- *(1) Sort (5)                  -- local sort, one per SMJ input
         :  +- Exchange hashpartitioning(route_id, 200) (4)  -- shuffle #1
         :    +- *(1) Filter (3)
         :      +- *(1) ColumnarToRow (2)
         :        +- BatchScan trips (1)    -- FileScan for a Hive table
      +- *(1) Sort (8)
         +- Exchange hashpartitioning(route_id, 200) (7)  -- shuffle #2
         +- BatchScan dim_route (6)

(1) BatchScan trips
  PartitionFilters: [event_ts >= 2026-03-01, event_ts < 2026-03-08]
  PushedFilters:    [IsNotNull(route_id)]          -- row groups skipped
  ReadSchema:       struct<route_id,arrival_delay_s,event_ts>

```

Listing 2.1: EXPLAIN FORMATTED on the specimen (trimmed)

The nodes you will meet most — a full catalogue is Appendix C:

Scan, at every leaf. BatchScan for an Iceberg or other DataSource V2 table; FileScan for a Hive-catalogued table; InMemoryTableScan for cached data. Its fields are where pruning is confirmed or denied: PartitionFilters (whole partitions skipped — directories on object storage), PushedFilters (row groups skipped inside the files that are read, using column min/max and bloom filters), DataFilters (everything, including the part evaluated row-by-row after reading), and ReadSchema (the columns actually read — should be only those the query needs). Batched: true means vectorized columnar reads are on.

Exchange, at every stage boundary. Each Exchange is one shuffle: serialize, write to local disk, transfer, re-read. Count them — it is the best single predictor of cost. hashpartitioning(key, n) co-locates equal keys (joins, GROUP BY); rangepartitioning

produces globally ordered partitions (a global `ORDER BY`); `SinglePartition` funnels everything to one task (a global aggregate, `coalesce(1)`). The `ENSURE_REQUIREMENTS` tag means the planner inserted this exchange because a downstream operator needs that distribution. `BroadcastExchange` is a different move: it ships one small side whole to every executor, and it appears only below the build side of a `BroadcastHashJoin`.

Sort. `Sort [key ASC NULLS FIRST], false, 0` — the boolean is true for a global sort, false for a local per-partition one. A `SortMergeJoin` always has two local sorts, one per input, between the exchanges and the join.

Tip

The fast scan of any plan, in order: (1) the join operator — `SortMergeJoin` on a small dimension means missing statistics (Chapter 5); (2) the number of `Exchange` nodes; (3) `PartitionFilters` non-empty on a partitioned table (Chapter 7); (4) `PushedFilters` non-empty; (5) `ReadSchema` listing only needed columns.

After the query runs, the SQL tab shows the same tree with *actual* row counts and, if AQE was active, the adapted plan rooted at `AdaptiveSparkPlan` `isFinalPlan=true` — which is the plan that really ran, not the one `EXPLAIN` printed beforehand (Chapter 6).

2.7 Summary

A query becomes an execution in fixed stages: parse to a syntax tree, resolve names and types against the catalog, rewrite the logical plan with mostly heuristic rules (pushdown, column pruning, folding; join reordering only with statistics), choose physical operators and shuffle points with strategies, and generate Java for each pipelined chain. The physical plan — readable with `EXPLAIN FORMATTED` — tells you the join strategy, the number of shuffles, and whether pruning and column selection are working, which together determine how many stages the query has and where it will spend its time. The recurring lakehouse failure is missing catalog statistics, which makes the cost-based decisions (join strategy and order) unreliable until you `ANALYZE` or override.

Exercises

1. Run `EXPLAIN FORMATTED` on the specimen query. Confirm the two Exchange nodes, the SortMergeJoin, and a non-empty PartitionFilters on the trips scan.
2. Write the same result three ways: with an explicit subquery, with a CTE, and as one flat statement. Compare the *optimized* logical plans. Which differ, and which are identical?
3. Add `WHERE CAST(t.route_id AS STRING) = '47'` to the query and re-run `EXPLAIN FORMATTED`. What happened to PushedFilters?

Chapter 3

How an Executor Uses Memory and CPU

Spark runs on the Java Virtual Machine: the driver and every executor is a JVM process, and the engine's memory behaviour, its failure modes, and much of its internal design follow directly from how the JVM manages memory and runs code. This chapter builds the memory map of a single executor — which regions exist, which a query can use, and which are consumed no matter what — then adds the two costs layered on top of it: garbage collection and serialization. By the end you can size a pod from its configuration, explain a container kill that left no Java error, and read GC time in the UI. How execution and storage memory borrow from each other while a job runs is picked up alongside caching in Chapter 8; how to size these regions for a workload is Chapter 14.

3.1 The JVM in one page: bytecode and just-in-time compilation

Spark is written in Scala and runs on the Java Virtual Machine, and two features of that platform shape how it performs.

The first is *just-in-time compilation*. Scala compiles to JVM *bytecode*, a portable instruction set the JVM does not run directly. Instead it interprets bytecode at first and

watches which methods run often; a *hot* method is then compiled to native machine code by the JIT, with optimizations — inlining, loop unrolling, dead-code elimination, escape analysis — that depend on what the method actually did the first few thousand times it ran. Two consequences follow. A Spark process is slowest in its first seconds, before the JIT has warmed up, which is one reason Kyuubi keeps a driver alive rather than starting one per query (Section 1.3) and why a micro-benchmark that runs a snippet once tells you almost nothing. And the JIT compiles *tight, predictable* code far better than code full of virtual calls and branches — which is exactly why Spark’s execution engine generates a single flat Java method for a chain of operators (Section 2.5, Chapter 17) instead of walking an operator tree.

The second is *automatic memory management*: the program allocates objects and never frees them, and a *garbage collector* reclaims what is no longer reachable. This is a convenience with a running cost, and for a data engine moving billions of rows the cost is large enough that Spark works hard to keep most of its data *out* of the managed heap entirely. The rest of this chapter is about where an executor’s memory goes, and Section 3.4 about what the garbage collector does to it.

3.2 Heap layout and object representation

The JVM manages a single contiguous region of memory, the *heap*, from which every object a program allocates is drawn. A Spark executor is one JVM process, and the value of `spark.executor.memory` is precisely the maximum size of that heap. Everything a task does — decoding a Parquet page into rows, building a hash table for a join, accumulating a group-by result — competes for space in it, and when it fills, the task must either spill to disk or fail.

Two properties of the heap drive most of Spark’s decisions about memory. The first is *per-object overhead*. Every object carries a header — a mark word and a pointer to its class — before any field is stored, and the allocator pads every object to an eight-byte boundary.

```
java.lang.Integer
  object header    12 bytes    (mark word + class pointer)
  int value        4 bytes
```

```

-----
total          16 bytes   to carry 4 bytes of data

```

Listing 3.1: Cost of a boxed integer on a 64-bit JVM

A row represented as an array of boxed Java objects (an `Integer`, a `Double`, a `String` with its own header and backing character array) can be several times larger in the heap than the same row encoded as raw bytes. This is the amplification you see when a compressed Parquet file expands to many times its on-disk size once decoded into Java objects, and it is why Spark’s execution engine stores rows off the object heap in a packed binary format rather than as JVM objects (Chapter 17).

The second property is that the heap is *managed*: the program never frees memory explicitly, and reclamation is the job of the garbage collector (Section 3.4). Spark does not hand the whole heap to a query; it subdivides it into regions with different owners and different rules, described next.

3.3 The executor memory model

Configuring Spark’s memory means deciding how one executor process divides the memory it is allowed to use. That process holds three separately configured pools, and confusing them is the most common source of both wasted memory and unexplained container kills.

Analogy

Picture the heap as an apartment building with a fixed number of floors (`spark.executor.memory`). One utility closet is permanently locked for building systems (*reserved*) — no tenant ever gets a key. A shared common floor (*unified memory*) is reconfigured on demand: some nights it hosts a big event that needs the whole floor (*execution*), other nights it is rented out as storage lockers (*storage*) — and management can always cancel a storage booking to make room for a scheduled event, but never the reverse. The remaining floors are private apartments (*user memory*): whatever a tenant does inside their own apartment is invisible to the building manager, and if a tenant packs it past capacity, nobody bails them out — the apartment simply

collapses (`OutOfMemoryError`). A separate storage unit down the street, rented by some tenants but not others, is the *off-heap* pool; and the building's own infrastructure — elevators, wiring, hallway lighting (*overhead*) — draws power regardless of occupancy. The whole lot only fits inside the property line the landlord (the cluster manager) enforces: exceed it, and the building is condemned, tenants and all.

The heap. `spark.executor.memory` sets the maximum size of the JVM heap, the region every object the executor allocates comes from. Spark carves it into three parts:

- **Reserved memory** — a fixed 300 MB Spark holds back for its own internal objects. It is not configurable and never available to a query.
- **Unified memory** — by default 60% of (heap – 300 MB), set by `spark.memory.fraction`. This is the only pool Spark accounts for byte by byte and the only one that spills to disk gracefully when it runs out. It is shared between *execution memory* (the working memory of a running operator: hash tables for joins and aggregations, sort buffers, the shuffle write buffer) and *storage memory* (cached `DataFrame` and `RDD` blocks, and broadcast data held on the executor). `spark.memory.storageFraction` (default 0.5) is a soft boundary between the two: either side may borrow the other's idle space, but execution can always evict borrowed cached blocks to reclaim its half, whereas storage cannot push execution below the border. How this plays out while a job runs is covered in Chapter 8.
- **User memory** — the remaining 40%. Everything Spark does not manage lives here: the objects your UDFs and `RDD` closures create, the structures a `mapPartitions` builds up. Spark does not track this pool and cannot spill it; overflowing it throws `OutOfMemoryError` and kills the executor.

Off-heap memory. When `spark.memory.offHeap.enabled` is set, `spark.memory.offHeap.size` adds a second unified-memory pool that lives outside the heap, addressed directly by the engine and invisible to the garbage collector (Section 3.4). Execution and storage draw on it in addition to their on-heap pool. It is not taken out of `spark.executor.memory`; it is additional memory the process reserves, and it must be counted into the container's limit.

Overhead. `spark.executor.memoryOverhead` (default the larger of 384 MB and 10% of the heap) is memory the process needs that is neither heap nor off-heap pool: JVM thread stacks, class metadata (Metaspace), buffers for network transfer and compression codecs, and — for PySpark — the Python worker processes, which can each take hundreds of megabytes. It is not a safety margin; it is space that is actually consumed, and its default is frequently too small for PySpark or codec-heavy workloads, in which case the container is killed with no Spark-level error.

What the cluster manager enforces. The pod or container is sized as the sum of all three pools (Figure 3.1). If the process’s real usage crosses that limit — most often because overhead was underestimated or user memory grew unbounded — the kernel kills it (Section 15.4). The heap merely filling up, by contrast, makes unified memory spill to disk and only throws when user memory or a single un-spillable allocation is the cause.

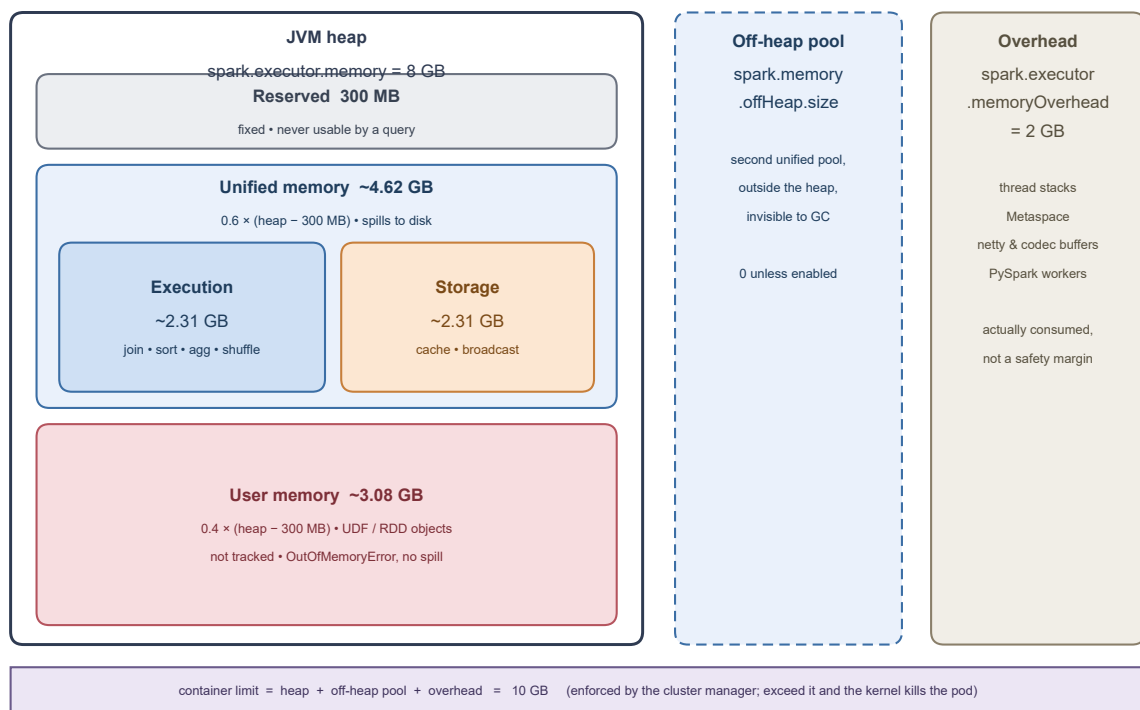


Figure 3.1: An executor’s memory: the JVM heap (reserved / unified / user), the optional off-heap pool, and the overhead the process needs outside both. The cluster manager’s container limit must cover all three. Numbers are worked for an 8 GB heap with 2 GB overhead.

Per task, not per executor. Execution memory is shared equally across the tasks an executor runs at once, which is `spark.executor.cores`. A larger heap with proportionally more cores gives no single task more room: memory per task is the execution pool divided by the core count. The number of cores per executor is therefore a memory decision as much as a parallelism one, and an executor with too many cores spills on work a narrower one finishes in memory (Section 14.1).

3.4 Garbage collection: Parallel, G1, ZGC

Because the heap is managed, the JVM must periodically find objects that are no longer reachable and reclaim their space. This work is the *garbage collector* (GC), and for a data-processing workload it is never free: the collector competes with the application for CPU, and some of its phases are *stop-the-world*, meaning every application thread in the JVM is frozen until the phase completes. On an executor a long stop-the-world pause stalls every running task at once, and if it exceeds the heartbeat timeout the driver declares the executor lost and re-runs its work elsewhere (Section 15.4).

Collectors differ mainly in how they trade pause length against throughput and CPU cost.

Analogy

Picture three ways to clean a large office building overnight. Parallel GC is an all-lights-off deep clean: every employee is sent home, the whole crew cleans the whole floor at once, and the doors reopen only when they are finished — the fastest total clean, but nobody works while it happens. G1 cleans one wing at a time while the rest of the building stays open, closing only the wing being cleaned right now, briefly and often. ZGC has cleaners who tidy around the working employees continuously and almost invisibly, closing nothing at all — at the cost of moving more carefully and needing a bigger crew.

- **Parallel GC** uses multiple threads but collects the whole heap in one stop-the-world pause. It gives the highest raw throughput when pauses do not matter, but pause time grows with heap size, so a large executor can freeze for seconds at a time.

- **G1** (Garbage-First) divides the heap into equal-sized regions and collects a subset of them per cycle, doing most of its marking concurrently with the application. Pause times are shorter and roughly bounded regardless of heap size, at the cost of more bookkeeping and CPU. It is the default at the heap sizes a Spark executor typically runs and the collector most Spark tuning assumes.
- **ZGC** does essentially all of its work concurrently and keeps pauses under a millisecond even on very large heaps, at a higher constant CPU and memory overhead. It is worth choosing only for executors with tens of gigabytes of heap where G1 pauses are still a problem.

Collector	Pause	Throughput / CPU	Best for
Parallel	long, grows with heap	highest throughput, lowest overhead	batch jobs where pauses don't matter
G1 (default)	short, roughly bounded	moderate bookkeeping cost	most Spark executors
ZGC	sub-millisecond	higher constant CPU & memory	very large heaps where G1 still pauses

Table 3.1: The three collectors, at a glance.

The generational hypothesis — that most objects die young — underlies all three: they collect a small “young” area cheaply and often, and the full-heap collection rarely. A workload that holds a large amount of long-lived data in cache violates that assumption and pushes the collector toward the expensive path, which is one more reason to cache deliberately (Chapter 8). Monitoring GC time is covered in Section 15.3; the executor JVM options that select and tune a collector are in Chapter 14.

3.5 Serialization and its cost

The engine keeps a join’s hash table or a block of cached rows as one large off-heap byte buffer rather than millions of small Java objects, which is why that data is invisible to

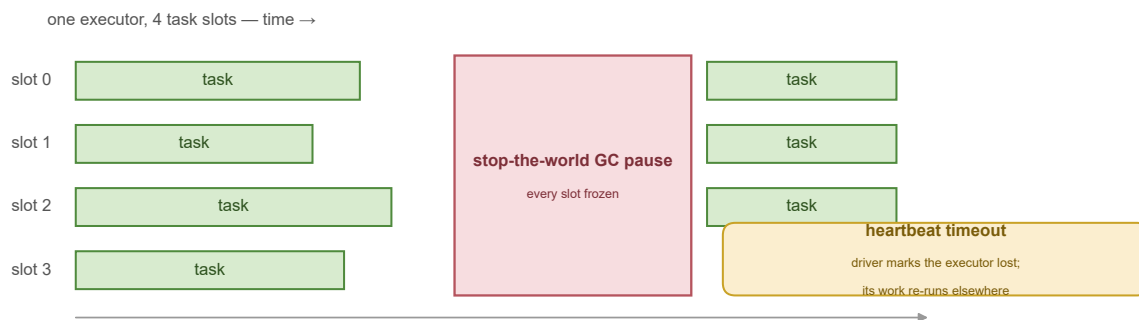


Figure 3.2: A stop-the-world pause freezes every task slot on the executor at once, not one at a time. If the pause outlasts the heartbeat interval, the driver concludes the executor is gone and re-runs its work elsewhere — so a GC problem shows up as lost executors and repeated stages, not just as slow ones.

the garbage collector and costs nothing to collect (Section 3.3, Chapter 17). Getting data into and out of that flat form is serialization.

Serialization is the conversion of an in-memory object graph into a flat sequence of bytes, and *deserialization* is its inverse. Spark serializes whenever data leaves a JVM: across the network during a shuffle, to disk when unified memory spills or a block is cached in serialized form, and when a broadcast variable is shipped to executors. The cost is both CPU and size.

The two options for arbitrary Java objects differ sharply. The built-in Java serializer writes the full class name and every field name and type alongside the values, repeated for every object, so most of the output is self-description rather than data. *Kryo* writes a compact encoding — a short numeric class identifier and the field values — and produces output several times smaller, which directly reduces shuffle volume, spill size, and broadcast footprint (Chapter 14).

DataFrame and SQL workloads sidestep both serializers for most operations: rows already live in the engine’s binary format (Chapter 17), so moving them is a memory copy with no object graph to walk. The Java-versus-Kryo choice still governs RDD code, user-defined functions, and anything that puts arbitrary Java objects into a shuffle

or a broadcast — the same user-memory objects that Section 3.3 noted Spark does not manage.

3.6 Summary

An executor is one JVM process, and its memory is three separately configured pools: the heap (`spark.executor.memory`), an optional off-heap pool, and overhead. The heap is carved into a fixed 300 MB reserve, *unified memory* (the only pool Spark accounts for and the only one that spills), and *user memory* (untracked, un-spillable, throws on overflow). The container limit the cluster manager enforces is the sum of all three; cross it and the process is killed with no Java error, whereas merely filling the heap makes unified memory spill. Execution memory is shared across an executor's cores, so cores-per-executor is a memory decision. On top of the map sit two costs: garbage collection, whose stop-the-world pauses freeze every task at once and can cost you an executor; and serialization, which Kryo makes several times cheaper than the Java default for RDD and UDF code, though DataFrame rows bypass it entirely. You can now size an executor from its workload and explain the two kinds of memory failure by which pool overflowed.

Exercises

1. Given a pod memory limit and a workload description, propose `spark.executor.memory`, `spark.executor.cores`, and `spark.executor.memoryOverhead`, and justify each number from the memory map in Figure 3.1.
2. An executor is killed by Kubernetes with `OOMKilled` and no Spark exception in the log. A second run fails with a Java `OutOfMemoryError` in a task. Which pool overflowed in each case, and which knob addresses each?
3. In the Executors tab, one executor shows 40% of task time in GC while the others show 5%. What is the most likely cause, and which chapter covers the fix?

Part II

Making Queries Fast

Chapter 4

The Shuffle

Every JOIN, GROUP BY, DISTINCT, window function, and repartition puts a wall between two stages: the shuffle. It writes every partition to disk, moves it across the network, and re-reads it. It is where most of a query's time goes, and it is the mechanism behind the wide dependency from Section 1.5. This chapter opens the wall: the write path, the read path, how to size shuffle partitions from data volume instead of the default 200, and how to find and remove exchanges the query does not need.

4.1 What a partition is: input partitions versus shuffle partitions

Spark never processes a table as one unit; it splits it into **partitions**, and one task handles exactly one partition (Section 1.4). Two different mechanisms create them, and mixing them up is a common source of confusion when reading a plan.

An **input partition** is created when a stage reads from storage. Spark decides how many by dividing the bytes to read by a target size: roughly `spark.sql.files.maxPartitionBytes` (128 MB) for Hive-catalogued files, or the split size Iceberg's planner reports from its manifests (`read.split.target-size`, also 128 MB by default, Chapter 12). One data file is not one partition either way — small files are combined and large ones split so that each input partition is close to the target size.

Note

Both configs measure the file's *on-disk* (compressed) length, never the decompressed size — neither Spark nor Iceberg opens a file just to plan splits. For a compressed columnar file this is the same amplification as Section 3.2, one step earlier: a “128 MB” partition can decode to several times that in a task's execution memory, a quiet cause of spill unrelated to shuffle-partition sizing.

Splittability is a separate question from ratio: Parquet and ORC stay splittable at any codec, since Spark cuts at independently-decodable row-group or stripe boundaries. A .gz-compressed text/CSV/JSON file cannot — gzip's stream has no resume point, so the whole file is one partition regardless of size.

A **shuffle partition** is created when a wide operator redistributes data. Its count is set directly: by `spark.sql.shuffle.partitions` (default 200) for a join or aggregation, or by an explicit `repartition(n)`. Spark decides which shuffle partition a row belongs to with a hash:

```
partition_id = hash(key) mod num_shuffle_partitions
```

The same key always hashes to the same partition, which is exactly what a join or a `GROUP BY` needs — it is how rows that must be compared end up on the same executor. Note what this does *not* guarantee: a shuffle partition holds every row for potentially many different keys, and nothing about the hash keeps that mixture even. Two keys can collide into the same partition, and one key alone can dominate a partition if enough rows share it. That is the seed of data skew (Chapter 6); for now, the point is that shuffle-partition count is a number *you* choose, not one Spark infers from the data the way it infers input-partition count.

Analogy

A shuffle is a citywide moving day: many offices, each organized by building, have to be reorganized by department into new, single-department buildings. Every current building (a map task) has its people first check which new building they've been assigned to (compute `partition_id`), sort themselves into that order, and box up their belongings by destination — each building keeping one manifest of what it packed and where each box is going (the one data file plus its index). The mov-

ing company's dispatcher (the map output tracker) is the only party who ever holds the complete picture of which old building currently has which department's boxes. When a new department building is ready (a reduce task), it does not go door to door itself — it asks the dispatcher, then sends trucks to every old building that packed something for it (the all-to-all fetch), and once everything arrives it unpacks and arranges the furniture in the new layout (the merge). If a truck can only carry so much in one trip, the extra boxes wait on the sidewalk until the next one — a spill to disk.

4.2 The shuffle write path

Figure 4.1 follows one shuffle from a map task in stage 0 of the specimen to a reduce task in stage 2. On the write side, every map task:

1. computes `partition_id = hash(route_id) mod 200` for each of its output rows;
2. sorts its rows by that partition id (Spark has used only this *sort-based* shuffle writer since 1.6 — there is no separate hash-shuffle path to choose between any more);
3. spills the sort to local disk if the shuffle write buffer (bounded by Section 3.3's execution memory) fills before the task's data does;
4. writes the result as *one* data file plus *one* index file for the whole task, with the index recording where each partition's byte range begins.

One file per map task, regardless of how many shuffle partitions there are, is why raising `spark.sql.shuffle.partitions` does not multiply the number of files on disk — it only changes how the one file's byte ranges are cut up. The driver's *map output tracker* records, for every map task, which executor holds that file and the offsets for each partition inside it.

4.3 The shuffle read path

A reduce task starts by asking the map output tracker for the location of its partition's data across *every* map task in the previous stage — for stage 2 of the specimen, that is

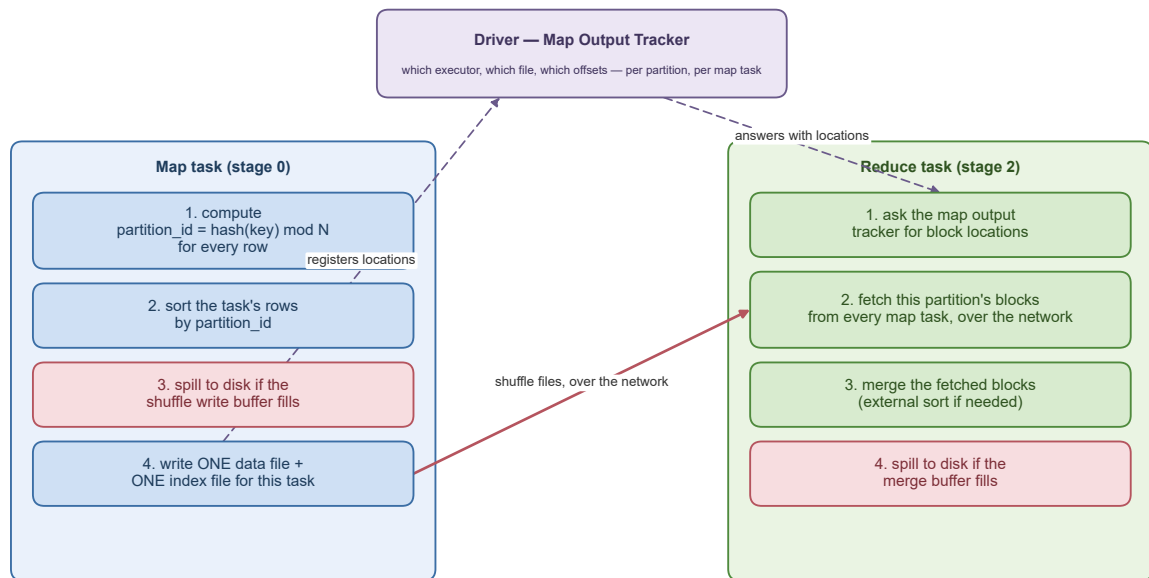


Figure 4.1: The shuffle write and read paths. The map task never talks to the reduce task directly: it registers its file’s location with the driver’s map output tracker, and the reduce task asks the tracker where to fetch from.

every one of stage 0’s and stage 1’s tasks. It then fetches those blocks over the network via the block transfer service, several fetches in flight at once up to `spark.reducer.maxSizeInFlight`, and merges them — an external (disk-backed) merge if the combined size does not fit in the task’s share of execution memory.

This is the step that makes a shuffle expensive relative to a narrow operation: it is genuinely all-to-all. With M map tasks and R reduce tasks, up to $M \times R$ blocks are produced and each of the R reduce tasks may fetch from each of the M executors that produced them. Section 4.6 covers the mechanism Spark added specifically to make this fetch pattern cheaper at scale; the scheduler machinery behind both sides is Chapter 18.

4.4 Sizing shuffle partitions

Because the write path always produces the same one file per task, the only real lever on shuffle cost is *how big each shuffle partition ends up*, which is set by `spark.sql.shuffle.partitions` against the total volume being shuffled:

```
target partitions = total shuffle bytes / target task size
(128–200 MB)
```

The default of 200 is not tuned to your data; it is just a constant. For a 100 GB shuffle it is badly wrong in one direction, as Figure 4.2 makes exact: 200 partitions puts 512 MB in every task, several times past the point where the task’s share of execution memory holds it and it spills to disk. A query that shuffles only a few gigabytes has the opposite problem — 200 partitions of a few megabytes each, so the fixed per-task scheduling overhead dominates the actual work.

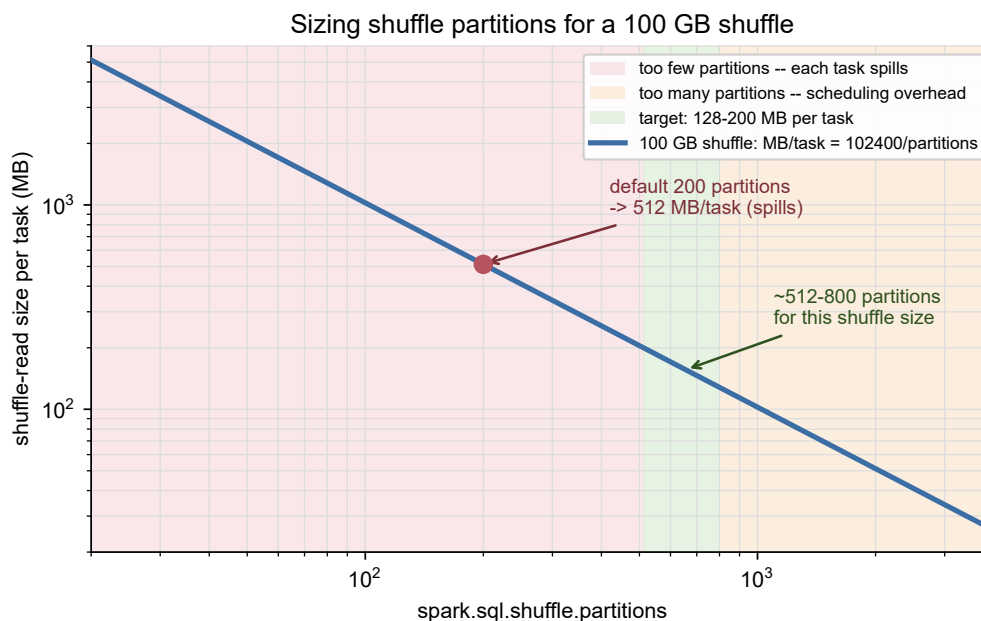


Figure 4.2: For a fixed shuffle size, size per task is exactly (shuffle bytes) / (partition count) — an exact relationship, not a rule of thumb. For a 100 GB shuffle the default 200 partitions puts 512 MB in every task, well past where it spills; 512–800 partitions lands each task in the 128–200 MB band.

Warning

If you set `spark.sql.shuffle.partitions` too low, Spark does *not* raise it for you. Partitions simply grow past the point where they fit in memory and spill, silently, until the job either finishes slowly or runs out of disk. There is no auto-correction on the low side.

The practical answer is to set the count generously as a ceiling and let Adaptive Query Execution shrink it back down once it has seen the real data size, rather than trying to predict the exact right number up front:

```
spark.sql.shuffle.partitions = 2000
```

```
spark.sql.adaptive.enabled                = true
spark.sql.adaptive.coalescePartitions.enabled = true
spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728
-- 128 MB
```

Over-partitioning is cheap to fix at runtime — AQE merges small adjacent partitions after the shuffle completes and it knows their true size (Chapter 6); under-partitioning, which spills, is not something a later stage can undo. Set high and let AQE come down, not the other way around.

Partition count versus the cluster’s task slots. The formula above sizes a partition in bytes; it says nothing about how many can run at once, which is set separately by the cluster’s total task slots:

```
total task slots = executor count x spark.executor.cores
```

Partition count against *that* number, not against the shuffle size, is what Section 1.4 meant by “8 tasks on a cluster with 200 free slots leaves 192 idle” — too few partitions wastes capacity you are already paying for. The reverse failure is subtler: if partition count is only a little more than a multiple of the slot count — 41 partitions on 40 slots, say — the first wave uses every slot, and the second wave runs on a single core while the other 39 sit idle, costing nearly as much wall-clock time as the first wave for a fraction of the work. Prefer a partition count that divides the slot count cleanly over one that merely clears it.

Executor count is not a fixed input here either. Under dynamic allocation (Section 14.2), Spark requests more executors when it sees more pending tasks than running ones, so a higher partition count can grow the cluster rather than the other way around — the causal arrow runs from partitions to executors, not from a fixed executor count down to how many partitions to choose. What executor *shape* does constrain is partition *size*: execution memory per task is the execution pool divided by `spark.executor.cores` (Section 3.3), so more cores packed into the same executor leaves less room per task and calls for smaller partitions to avoid spill.

4.5 Shuffle compression

Shuffle data is compressed by default (`spark.shuffle.compress=true`) using `spark.io.compression.codec`, which defaults to `lz4`. That default is deliberate: shuffle bytes are written once and read back almost immediately by the very next stage, so *decompression speed* dominates the cost, not compression ratio. `lz4` and `snappy` both compress and decompress in a small fraction of the time `zstd` or `gzip` take, at a somewhat worse ratio. That trade-off is the opposite of Parquet storage on Iceberg, where a file is written once but may be read by hundreds of later queries, so a slower, denser codec such as `zstd` pays for itself (Chapter 11). Using the same codec for both because “it worked for storage” is a common way to leave shuffle performance on the table.

Codec	Ratio	Compress	Decompress	Best for
lz4	low–medium	very fast	very fast	shuffle (the default)
snappy	low–medium	very fast	very fast	shuffle, an alternative to lz4
zstd	high	medium	fast	Parquet storage on Iceberg
gzip	highest	slow	slow	archival only — avoid for shuffle

Table 4.1: Written once, read immediately (shuffle) wants decompression speed; written once, read many times (storage) can afford to spend more on ratio.

4.6 Push-based shuffle

The read path in Section 4.3 has a structural cost at scale: many small, random fetches, one per (map task, reduce partition) pair, which stresses the block transfer service and, if a map executor is lost, forces its tasks to be recomputed before the shuffle can finish. *Push-based shuffle* (stable since Spark 3.2) restructures this: after writing its file, each mapper also *pushes* its blocks to shuffle services, which merge blocks from many mappers that belong to the same reduce partition into a small number of larger merged files. A reduce

task then does a handful of large sequential fetches instead of many small ones.

Note

Push-based shuffle needs an external, out-of-executor shuffle service capable of receiving and merging pushed blocks, which is a natural fit for YARN’s long-lived Node Managers. Its support on Kubernetes is narrower and depends on running a compatible external shuffle service alongside your executors — confirm it is available on your cluster (Appendix B) before turning on `spark.shuffle.push.enabled`; without it, the setting has nothing to push to.

4.7 Finding and removing unnecessary exchanges

Every Exchange in a physical plan is a shuffle you are paying for (Section C.4), and not every one is necessary. Four patterns account for most avoidable ones.

A manual `repartition()` right before an Iceberg write. If the target table has `write.distribution-mode=hash` (Chapter 13), Iceberg already shuffles the write by partition key. Calling `repartition()` yourself first adds a *second*, usually redundant, shuffle. Let the table’s distribution mode do it, or set the mode to `none` only when you are certain the data is already correctly distributed.

A `DISTINCT` on data that is already unique. `DISTINCT` compiles to an aggregation, which shuffles. If the upstream source already guarantees uniqueness — a primary key, or a prior `GROUP BY` on the same columns — the `DISTINCT` is pure cost.

An `ORDER BY` inside a subquery. Only the outermost `ORDER BY` in a query is guaranteed to survive to the result; SQL does not promise row order through an intermediate view or CTE. An inner sort still costs a range-partitioning exchange even though nothing downstream can rely on its effect.

Repeated aggregation on the same key. Two separate `GROUP BY route_id` computations in the same query — one for the on-time rate, one for total trips — shuffle twice on the same key when a single `GROUP BY` with two aggregate functions would shuffle

once. Watch for `ReuseExchange` in the plan (Section C.4): the planner catches this only when the two exchanges are byte-for-byte the same partitioning, so a small difference (a different partition count, an added column) silently defeats it.

Tip

Run `EXPLAIN FORMATTED` and count the `Exchange` nodes before touching anything else. Every one you remove is a shuffle the job no longer pays for; every one you cannot explain is worth a second look.

4.8 Summary

A shuffle partition is not the same thing as an input partition: input partitions come from splitting storage by size, shuffle partitions come from a count you set and a hash that routes rows into it. The write path always produces one file per map task, indexed by partition; the read path is genuinely all-to-all, which is what makes a shuffle costly. Size `spark.sql.shuffle.partitions` from the actual volume being shuffled — 128–200 MB per task is the target — set it generously and let AQE coalesce down rather than risk under-partitioning, which spills and does not self-correct. `lz4` trades ratio for decompression speed, which is the right trade for data read once, immediately. Push-based shuffle turns many small fetches into few large ones where an external shuffle service supports it. And the cheapest shuffle is the one you remove: count the `Exchange` nodes and look for a redundant repartition, `DISTINCT`, inner sort, or duplicated aggregation before reaching for any other fix.

Exercises

1. From a stage's shuffle-read size in the UI, compute a target partition count for a 128 MB task size, then verify against the observed task sizes.
2. Run the specimen query with `spark.sql.shuffle.partitions` set to 4000 instead of 200. Confirm in the UI that stage 2 now has 4000 tasks, and that Figure 4.2's formula predicts their size correctly.
3. Find a query in your own workload with two `Exchange` nodes that partition on the

same column. Check whether `ReuseExchange` fired; if not, find the difference between the two exchanges that prevented it.

Chapter 5

Joins

Spark has four ways to run a join and picks one from a size estimate. When the estimate is right, the choice is right. When the table has no statistics — the common case for Iceberg, as the specimen query already showed (Section 2.2) — the planner guesses, and a join that should have broadcast a 30 MB dimension instead shuffles a billion rows. This chapter covers the four strategies, how the planner chooses between them, and how to override it when you know better than the statistics do.

Figure 5.1 shows the shape each strategy leaves in a plan. The rest of this chapter explains why each shape exists and what it costs; the identifiers to spot them by are catalogued in Section C.6.

5.1 Broadcast hash join

If one side of a join is small enough, Spark skips shuffling the large side entirely: it collects the small side to the driver, packages it once, and ships a full copy to every executor (a `BroadcastExchange`), where it is built into an in-memory hash table keyed on the join column. Each task on the large side then probes that local hash table row by row — no shuffle, no sort, and the large side is read exactly once, in place.

This is the cheapest strategy whenever it applies, which is exactly why the planner tries it first. Two costs are easy to underestimate. First, the *driver* holds the whole broadcast table in memory while assembling it, at roughly twice the table’s serialized size; a

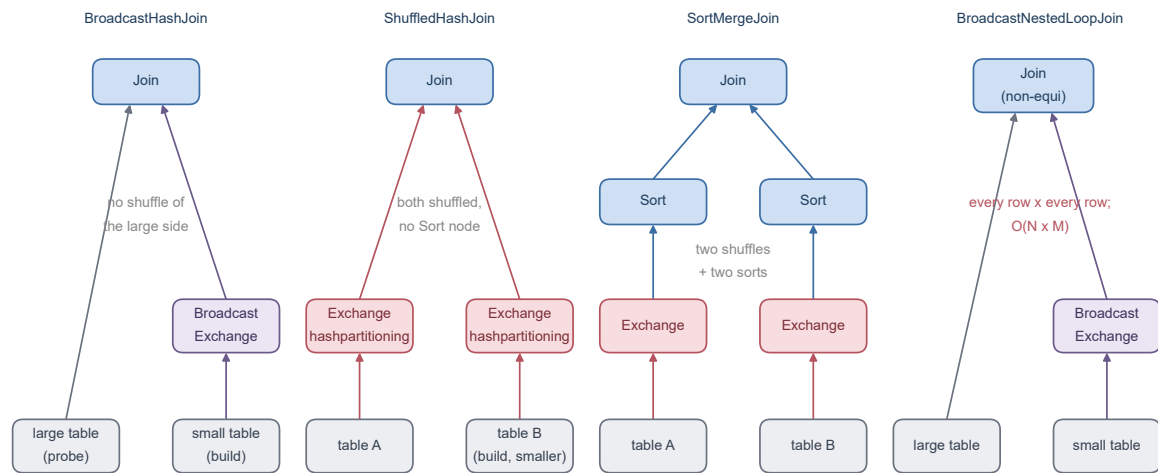


Figure 5.1: The four join strategies as plan shapes. Broadcast hash join shuffles nothing on the large side; shuffled hash join shuffles both sides but sorts neither; sort-merge join shuffles and sorts both; broadcast nested loop join broadcasts one side to evaluate a condition that is not an equality.

broadcast large enough to hurt the driver is a self-inflicted wound distinct from anything happening on the executors. Second, the size used to decide broadcastability is the table’s *on-disk* size, and a compressed columnar file routinely decodes to several times that once it is rows in memory (Section 3.2) — so a “100 MB” table can be genuinely 500 MB–1 GB once broadcast and decoded, which is why the planning-time threshold is conservative by design. Spark also enforces a hard ceiling (roughly 8 GB serialized, and a few hundred million rows) past which a table cannot be broadcast at all, regardless of the threshold.

5.2 Shuffle hash join

When neither side is small enough to broadcast but one side is still meaningfully smaller than the other, Spark can shuffle both sides on the join key and build an in-memory hash table from the smaller (build) side on each executor, then probe it with the larger (probe) side — without sorting either. This is a `ShuffledHashJoin`: two shuffles, like sort-merge, but no `Sort` nodes, which makes it cheaper than sort-merge whenever the build side reliably fits in memory per partition.

That condition is also its weakness: unlike sort-merge, a shuffled hash join has no graceful way to spill an oversized build-side partition to disk. If a single partition's build side is larger than expected — most often because of skew — the task fails outright rather than slowing down. Spark's planner is correspondingly conservative about choosing it: by default it prefers sort-merge whenever both sides have sortable keys (`spark.sql.join.preferSortMergeJoin=true`), and only considers a shuffled hash join when that preference is turned off and the build side is estimated at at least three times smaller than the probe side.

5.3 Sort-merge join

Sort-merge join is the default for two large, roughly comparable inputs, and it is what the specimen query uses in Chapter 2: both sides are shuffled on the join key, each partition is then sorted locally, and the join is produced by walking both sorted streams together, advancing whichever key is behind. It is the most expensive of the equi-join strategies — two shuffles and two sorts — but it is also the most robust: a sort spills to disk cleanly under memory pressure the way a hash table cannot, so it scales to inputs of any size on either side.

5.4 Broadcast nested loop join

An equi-join needs a hash or a sort; a join whose condition is not an equality — `ON a.price > b.min_price`, for instance — needs neither, and Spark falls back to comparing every row of one side against every row of the other. If one side is small enough to broadcast, this is a `BroadcastNestedLoopJoin`: no shuffle, but $O(N \times M)$ comparisons inside each task. If neither side is small enough to broadcast, Spark falls all the way back to a `CartesianProduct` (Section C.6), which is rarely what you want on two large tables and is usually a sign that a join condition is missing.

Note

A full outer join cannot use broadcast hash join, even with an explicit hint: there is no way to detect an unmatched row on the broadcast side without scanning it, which

the strategy is built to avoid. A `FULL OUTER` join falls back to sort-merge regardless of size.

5.5 How the planner chooses

For an equi-join, the planner tries strategies in a fixed order, stopping at the first one that applies:

1. an explicit join hint, if the query has one (Section 5.6);
2. **broadcast hash join**, if either side's estimated size is at or below `spark.sql.autoBroadcastJoinThreshold` (10 MB by default);
3. **sort-merge join**, if `spark.sql.join.preferSortMergeJoin` is true (the default) and the keys are sortable;
4. **shuffled hash join**, if that preference is off and the build side is estimated at least $3\times$ smaller than the probe side and fits per-partition memory;
5. **sort-merge join**, as the fallback for any remaining equi-join with sortable keys.

Every one of these decisions, except the explicit hint, is made from *estimated* sizes — from the catalog if statistics exist, or from on-disk file size if they do not (Section 2.2). This is the single most common way real joins go wrong: not a bug, but a plan built on a wrong number.

5.6 Forcing a strategy

When the estimate is missing or wrong, a hint overrides it directly:

```
1 SELECT /*+ BROADCAST(r) */ t.route_id, r.route_name
2 FROM trips t JOIN dim_route r ON t.route_id = r.route_id;
```

Listing 5.1: Forcing a broadcast when statistics are missing

The hints, in priority order, are BROADCAST (or BROADCAST JOIN/MAP JOIN), SHUFFLE_MERGE (or MERGE) for sort-merge, and SHUFFLE_HASH for a shuffled hash join. A hint is an instruction, not a suggestion: Spark will honor it even when its own estimate disagrees, so a stale hint on a table that has since grown past a sane broadcast size is a real production hazard, not a theoretical one.

The more durable fix is to give the planner real numbers instead of overriding its guess:

```
1 ANALYZE TABLE db.dim_route COMPUTE STATISTICS FOR ALL COLUMNS;
```

Listing 5.2: Giving the CBO real statistics

Iceberg tables carry file-level size and row counts in their metadata for free, but not the column-level statistics the cost-based optimizer uses to size a join — which is exactly why the specimen query’s small `dim_route` table was shuffled instead of broadcast in Chapter 2. Running `ANALYZE` once after a table’s data changes materially is usually cheaper, and safer over time, than hard-coding a hint.

Tip

Adaptive Query Execution catches many of these mistakes without any of the above: it re-evaluates a join’s strategy *after* its inputs have shuffled, against their real post-shuffle size, and promotes a sort-merge join to a broadcast if the data turns out small enough (Chapter 6). Hints and `ANALYZE` matter most for the joins AQE cannot rescue — most often because the small side is broadcastable only before a shuffle changes its size, not after.

5.7 Storage-partitioned joins for bucketed Iceberg tables

Every strategy above assumes the two sides arrive unaligned and have to be shuffled into agreement. If both tables are already *bucketed* on the join key with the same bucket count (Chapter 11), that assumption is false: rows that must be compared are already co-located file by file, and Spark can join corresponding buckets directly with no exchange on either side — a *storage-partitioned join*. This turns the most expensive part of a sort-merge join, the two shuffles, into work done once at write time instead of on every query that joins the tables.

The condition is strict: both tables need the identical bucket transform and bucket count on the join column, and Spark needs `spark.sql.sources.v2.bucketing.enabled` set to consider it at all. It is a design decision made when the tables are created, not a query-time tuning knob — worth it specifically for a join pattern repeated often enough that paying the shuffle cost once, at write time, beats paying it on every read.

5.8 Summary

Four strategies, in the order the planner tries them: broadcast hash join (cheapest, but bounded by the small side’s real decoded size, not its file size), sort-merge join (the robust default for two large sides), shuffled hash join (cheaper than sort-merge when it applies, but unable to spill), and broadcast nested loop or Cartesian for a non-equi condition. Every automatic choice rests on a size estimate, and Iceberg tables commonly lack the column statistics that estimate needs — `ANALYZE TABLE` and join hints exist specifically to correct that, and AQE corrects a real subset of it for free at runtime. Bucketing both sides of a frequent join on the same key removes the shuffle from the decision entirely.

Exercises

1. Given four table sizes and the broadcast threshold, predict each join’s strategy. Then change one config value and predict which join flips.
2. Run the specimen query, then run `ANALYZE TABLE dim_route COMPUTE STATISTICS FOR ALL COLUMNS` and run it again. Did the join strategy change? Explain from Section 5.5.
3. Write a join with a non-equi condition on two tables, neither of which is small. Confirm in `EXPLAIN FORMATTED` that it becomes a `CartesianProduct`, and rewrite it to avoid one.

Chapter 6

Adaptive Query Execution

The Catalyst optimizer plans before it has seen any data. Adaptive Query Execution re-plans after each shuffle, when the real partition sizes are known. It coalesces hundreds of tiny partitions into a few good ones, promotes a sort-merge join to a broadcast when the shuffled side turns out small, and splits a skewed partition so one task stops holding up the stage. This chapter covers what AQE re-decides, from what evidence, and which planning-time decisions it can still override.

6.1 Why runtime beats planning time

Every decision in Chapter 2 and Chapter 5 — how many shuffle partitions, which join strategy, whether to broadcast — is made from an *estimate*: a file size, a row count, a number the catalog may or may not actually have. Adaptive Query Execution is Spark inserting a checkpoint after every shuffle to replace a subset of those estimates with measurements. Once a shuffle has run, Spark knows the exact byte size of every resulting partition — no estimation error, no missing statistics, no compression-ratio surprise (Section 3.2) — and it uses that certainty to revise the plan for the stages that have not run yet. AQE has been on by default since Spark 3.2 (`spark.sql.adaptive.enabled`), and there is little reason to turn it off.

The trade AQE cannot make is the one that would help most: it cannot change a stage that has *already* run, and it cannot avoid a shuffle that planning-time decided was

necessary. It optimizes what happens after the measurement, not before it.

6.2 Coalescing shuffle partitions

Section 4.4 recommended setting `spark.sql.shuffle.partitions` generously and letting AQE bring it back down; this is the mechanism that does it. After a shuffle, AQE looks at the actual size of every partition and merges adjacent ones until each is close to `spark.sql.adaptive.advisoryPartitionSizeInBytes` (128 MB by default), stopping at `spark.sql.adaptive.coalescePartitions.minPartitionSize` (1 MB) so it does not over-merge. Two thousand shuffle partitions of a few megabytes each become a few dozen partitions of 128 MB, with no query change and no re-run.

Tip

Merging is one-directional. AQE only ever combines *adjacent* small partitions into fewer, larger ones — it never splits an oversized one this way. An oversized partition is handled by a different mechanism entirely (Section 6.5), and coalescing does nothing for it.

6.3 Converting sort-merge to broadcast at runtime

Section 5.5 chooses broadcast hash join from an estimate; AQE gets a second chance after the shuffle. If a sort-merge join’s shuffled input turns out, once measured, to be small enough, AQE replaces the sort-merge join with a broadcast hash join before the join actually runs — shown in the final plan as a `BroadcastQueryStage` in place of the exchange-and-sort pair (Section C.10). This rescues exactly the failure mode Chapter 5 described: an Iceberg table with no column statistics, planned as sort-merge because the planner had no better estimate, turns out small after its own shuffle, and AQE promotes it without you doing anything.

6.4 The two broadcast thresholds

`spark.sql.autoBroadcastJoinThreshold` and `spark.sql.adaptive.autoBroadcastJoinThreshold` are two separate configuration keys, and confusing them is

easy because they default to the same value. They are not measuring the same thing. The planning-time threshold is compared against a size estimate — on-disk bytes, before decompression (Section 3.2) — so it has to stay conservative to avoid broadcasting something that turns out several times larger once decoded. The adaptive threshold is compared against the *actual* size of already-shuffled, already-decoded Tungsten rows (Chapter 17): there is no amplification left to guard against, because the number is real. That is why it is reasonable to tune the two independently rather than assume one governs the other.

6.5 Skew-join handling: split and replicate

A shuffle partition can hold many keys unevenly (Section 4.1), and no amount of repartitioning fixes a single key that is simply larger than the others. AQE detects this after the shuffle: a partition is skewed if it is both larger than `spark.sql.adaptive.skewJoin.skewedPartitionFactor` ($5\times$ the median partition size) *and* larger than `spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes` (256 MB). Requiring both avoids flagging a partition as skewed just because the median happened to be tiny.

Figure 6.1 shows the fix. AQE splits the oversized partition into several sub-partitions and *replicates* the corresponding partition from the other side of the join across all of them — cheap, because the other side is small by comparison. Each sub-partition then runs as its own task, in parallel, instead of one task carrying the whole skewed key alone while every other task in the stage sits idle.

For skew severe enough that the defaults do not catch it, lowering both thresholds — `skewedPartitionFactor` to 3.0 and `skewedPartitionThresholdInBytes` to 128 MB — catches it earlier, at the cost of treating more ordinary variance as skew.

6.6 parallelismFirst and partition count

Coalescing toward a target size can go too far on a small cluster. If a shuffle produces 20 small partitions on a cluster with 16 cores, merging purely by size might collapse them to 2 partitions — leaving 14 cores idle for that stage even though there was enough data to

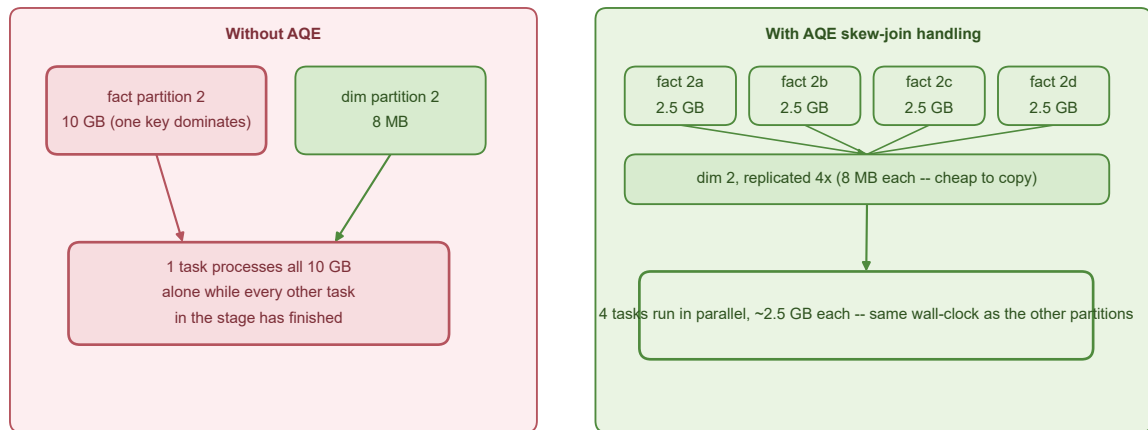


Figure 6.1: Without AQE, one oversized partition runs as a single task long after the rest of the stage has finished. AQE splits it into sub-partitions and replicates the small side across them, turning one straggler into several parallel, evenly-sized tasks.

keep all 16 busy. `spark.sql.adaptive.coalescePartitions.parallelismFirst` (default `true`) protects against this: AQE first checks how many partitions are needed to use every available core (Section 4.4’s task-slot arithmetic), and only merges down to the size target if that does not fall below it. Parallelism is kept even when it means some partitions stay smaller than the 128 MB advisory size.

6.7 Reading an AQE plan

The plan `EXPLAIN` prints before an action runs is not necessarily the plan that ran. The one that matters is the SQL tab’s plan after execution, rooted at `AdaptiveSparkPlan` `isFinalPlan=true`; the node types that appear only in that final plan — `AQEShuffleRead` `coalesced`, `BroadcastQueryStage`, `isSkew=true` — are catalogued with their exact plan snippets in Section C.10. Reading them is how you confirm which of this chapter’s optimizations actually fired, rather than assuming from the config alone.

6.8 Summary

Planning happens before any data moves; AQE runs after every shuffle, when partition sizes are exact rather than estimated, and revises three things: it coalesces small parti-

tions toward a target size (never splitting large ones by this mechanism), it promotes a sort-merge join to broadcast when the shuffled side is small enough by its own, more permissive threshold, and it detects a partition that is both far above the median and above an absolute size floor, splitting it and replicating the other side rather than leaving one task to carry it alone. None of this helps a stage that has already run or a shuffle that planning already committed to — which is why the earlier chapters’ advice (size shuffle partitions generously, give the planner real statistics, bucket a repeated join) still matters; AQE is the safety net under those decisions, not a replacement for them.

Exercises

1. Disable AQE and run a skewed join; record the straggler task. Re-enable it and compare the plan and the task-time distribution.
2. Force a sort-merge join on a table small enough to broadcast after its shuffle (Section 5.6’s `SHUFFLE_MERGE` hint still lets AQE act). Confirm in the final plan that AQE converted it to a broadcast anyway.
3. Given a stage with 20 shuffle partitions on a 16-core cluster, predict how far `coalescePartitions` will merge them with `parallelismFirst` on versus off.

Chapter 7

Partitioning, Bucketing, and Pruning

The cheapest data to move is data you never read. Partition pruning skips whole directories; dynamic partition pruning does it using values discovered from the other side of a join; bucketing lays data out so a join needs no shuffle at all. Each of these is easy to defeat — a function on the partition column, a `CAST`, a stray `SELECT *` — and the plan tells you when you have. This chapter covers each mechanism and how to confirm it is working.

7.1 Partition pruning and the pushdown killers

[Section to be written.]

7.2 Dynamic partition pruning

[Section to be written.]

7.3 Bucketing to avoid a shuffle

[Section to be written.]

7.4 repartition versus coalesce

[Section to be written.]

7.5 Column pruning and ReadSchema

[Section to be written.]

7.6 Summary

[Section to be written.]

Exercises

1. Write the same filter three ways — a function on the partition column, a CAST, and a plain literal — and check which keeps PartitionFilters populated in the scan node.

Chapter 8

Caching and Reuse

Caching keeps a result Spark would otherwise recompute — but the wrong cache wastes storage memory and starves execution, and a cache used once is pure overhead. Spark also reuses work you did not ask it to: an identical exchange or subquery computed once and read twice. This chapter covers when caching pays, which storage level to pick against the GC budget from Chapter 3, and how to see reuse in a plan. It is also where execution and storage memory finally meet at runtime.

8.1 When caching pays

[Section to be written.]

8.2 Lazy cache() versus eager CACHE TABLE

[Section to be written.]

8.3 Storage levels and the GC trade-off

[Section to be written.]

8.4 How execution and storage memory borrow at run-time

[Section to be written.]

8.5 Exchange and subquery reuse

[Section to be written.]

8.6 Unpersist and cache thrashing

[Section to be written.]

8.7 Summary

[Section to be written.]

Exercises

1. Build a query that reads one CTE twice. Show the plan reusing the exchange, then change the query so the reuse breaks and measure the difference.

Part III

The Iceberg Lakehouse

Chapter 9

Table Formats and the Iceberg Spec

An Iceberg table is not a directory of Parquet files with a manifest bolted on. It is a tree: a catalog pointer to a metadata file, to a manifest list, to manifests, to data and delete files — every level immutable, every commit a single atomic pointer swap. Reads consult this tree before touching any data. This chapter walks the spec (format version 2) against a real table's metadata tables so you can draw the tree from memory.

9.1 From Hive tables to table formats

[Section to be written.]

9.2 The Iceberg spec, format version 2

[Section to be written.]

9.3 Snapshots and the metadata tree

[Section to be written.]

9.4 Position deletes versus equality deletes

[Section to be written.]

9.5 Iceberg, Delta Lake, and Hudi

[Section to be written.]

9.6 Summary

[Section to be written.]

Exercises

1. Commit three appends to a table, then inspect `metadata_log_entries`, `snapshots`, and `manifests`. Explain how many metadata files and manifests exist and why.

Chapter 10

Catalogs and Commits

The catalog is the one component that turns a table name into the location of its current metadata — and the point at which a commit becomes atomic. Writing new metadata is not a commit until the catalog's pointer is swapped to it. When two writers race, one wins the swap and the other retries. This chapter covers what the Hive Metastore and REST catalogs do, how the atomic swap works, and how to tune retries so a busy table does not fail commits under contention.

10.1 What a catalog does

[Section to be written.]

10.2 The Hive Metastore catalog

[Section to be written.]

10.3 The REST catalog

[Section to be written.]

10.4 The atomic pointer swap

[Section to be written.]

10.5 Optimistic concurrency and retries

[Section to be written.]

10.6 Catalog configuration in Spark

[Section to be written.]

10.7 Summary

[Section to be written.]

Exercises

1. Run two writers into the same table. Observe the retry, then find the winning and losing commit attempts in the snapshot log.

Chapter 11

Iceberg Partitioning and Layout

Iceberg partitioning is hidden: you filter on `event_ts` and Iceberg derives the predicate on `day(event_ts)` for you, so a query never references a partition column directly and pruning does not break when the scheme changes. This chapter covers the partition transforms, evolving a spec without rewriting history, sort orders and Z-ordering, target file size, and the write-distribution mode — the choices that decide how much a reader and a writer have to move.

11.1 Hidden partitioning and partition transforms

[Section to be written.]

11.2 Partition evolution

[Section to be written.]

11.3 Sort orders and Z-ordering

[Section to be written.]

11.4 Target file size and the small-files problem

[Section to be written.]

11.5 Write distribution modes

[Section to be written.]

11.6 Summary

[Section to be written.]

Exercises

1. Given a daily data volume and a query pattern, choose a partition spec. Justify it against the target of 128 MB–1 GB per partition per write.

Chapter 12

Reading Iceberg Efficiently

Before the Iceberg reader opens a single data file, it has already skipped most of them — by partition, by manifest, by column statistics, and optionally by bloom filter. Each layer needs the query’s filter in a form it understands, and the scan report tells you how well each one worked. This chapter covers the pruning layers, the metrics modes that feed them, and how to read a scan report as a diagnostic.

12.1 Manifest pruning and metadata filtering

[Section to be written.]

12.2 Column statistics and predicate pushdown into Iceberg

[Section to be written.]

12.3 Metrics modes: full, truncate, counts, none

[Section to be written.]

12.4 Bloom filters

[Section to be written.]

12.5 Vectorized reads

[Section to be written.]

12.6 Split planning

[Section to be written.]

12.7 The metadata tables as a diagnostic

[Section to be written.]

12.8 Summary

[Section to be written.]

Exercises

1. Run a selective query and, from the scan report, compute the fraction of data pruned at each layer (partition, manifest, file).

Chapter 13

Writing and Maintaining Iceberg

Copy-on-write rewrites whole files on every update: slow writes, fast reads. Merge-on-read writes small delete files instead: fast writes, slower reads until compaction catches up. The choice is a function of your update rate and your read latency budget, and it commits you to a maintenance schedule — compaction, snapshot expiration, orphan-file cleanup — in a specific order. This chapter covers the write modes, optimizing MERGE, and running maintenance without deleting data a reader still needs.

13.1 Copy-on-write versus merge-on-read

[Section to be written.]

13.2 MERGE, UPDATE, and DELETE

[Section to be written.]

13.3 Write distribution and skew on write

[Section to be written.]

13.4 Compacting data files

[Section to be written.]

13.5 Rewriting manifests

[Section to be written.]

13.6 Snapshot expiration and orphan files

[Section to be written.]

13.7 Metadata management for streaming tables

[Section to be written.]

13.8 Summary

[Section to be written.]

Exercises

1. Run a MERGE with and without the partition column in the ON clause. Explain the difference in files scanned from the plan.

Part IV

Operating Spark

Chapter 14

Tuning Resources and Configuration

This chapter does not hand you a table of recommended settings. It derives an executor shape from the memory map in Chapter 3 and the shuffle in Chapter 4, so that every number — cores, heap, overhead, partition count — has a reason you can defend. It then covers dynamic allocation, the external shuffle service on Kubernetes, configuration precedence, and running Spark behind Kyuubi for many users. For the standalone list of tuning knobs with values, see `spark_memory_tuning.md` in the repository root.

14.1 Executor sizing from the mechanism

[Section to be written.]

14.2 Dynamic allocation

[Section to be written.]

14.3 Parallelism defaults

[Section to be written.]

14.4 The external and push-based shuffle service

[Section to be written.]

14.5 Configuration precedence and profiles

[Section to be written.]

14.6 Kyuubi and multi-tenancy

[Section to be written.]

14.7 Summary

[Section to be written.]

Exercises

1. Take a job that spills. Propose two different fixes — more memory versus fewer cores — and predict which the mechanism favours, and why.

Chapter 15

Debugging and Profiling

A slow job leaves evidence: in the SQL tab, the stages tab, the executor metrics, the event log, and — when you need it — a flame graph. This chapter teaches you to read that evidence in order, so that "the job is slow" becomes "stage 4 is skewed" or "the executors are GC-bound" in a few minutes. It also catalogs the failure signatures — OOM and container kills, fetch failures, stragglers — and what each looks like before you reach for a fix.

15.1 The Spark UI as a diagnostic

[Section to be written.]

15.2 Event logs and the history server

[Section to be written.]

15.3 Reading task, shuffle, spill, and GC metrics

[Section to be written.]

15.4 Common failure signatures

A job rarely fails as one incomprehensible error. It fails as one of a small number of recognizable signatures, each with its own evidence in the UI and its own fix.

OOM and container kills. *[Section to be written.]*

Fetch failures and stage retries. Shuffle files are not deleted as soon as they are read — Spark keeps them in case a later stage needs the same shuffle again, and cleans them up in the background as the driver’s JVM garbage-collects the objects that reference them (the *ContextCleaner*). In practice the deciding moment is simpler: that storage is local to the executor that wrote it (Section 4.2), and on the reference stack it is an `emptyDir` tied to the executor’s pod, so losing the pod — an OOM kill, a node drain, dynamic allocation scaling down — deletes its shuffle files immediately, with no way to recover them from that executor again.

A reduce task that tries to fetch from such an executor gets a `FetchFailedException`, visible in the driver log and the stage’s tasks tab. This is not a generic task failure: the DAG scheduler recognizes it specifically, marks that executor’s map output as gone in the map output tracker, and resubmits only the map tasks that ran on it — not the whole stage, and not the job — before retrying the reduce stage. You will see this in the UI as a stage that appears to run twice, with a smaller second attempt.

Tip

Two different layers retry, and tuning the wrong one does nothing. A fetch that fails from a flaky network but a still-live executor is retried by the shuffle client itself: `spark.shuffle.io.maxRetries` (3 attempts) and `spark.shuffle.io.retryWait` (5s between them). A fetch that fails because the executor is genuinely gone is handled by the stage resubmission above, bounded by `spark.stage.maxConsecutiveAttempts` (4) and, per task, `spark.task.maxFailures` (4) — raise both on a cluster where executors are routinely reclaimed (spot instances, aggressive dynamic allocation) rather than let the job fail after only a handful of retries. If the lost executors are *voluntary* removals under dynamic allocation (Section 14.2)

rather than crashes, `spark.shuffle.service.enabled` avoids the failure entirely by keeping shuffle data servable after its executor is gone (Section 4.6). Enabling `spark.excludeOnFailure.enabled` keeps subsequent retries from landing back on a node that is already causing them.

Stragglers. *[Section to be written.]*

15.5 async-profiler and flame graphs

[Section to be written.]

15.6 Debugging query plans

[Section to be written.]

15.7 Summary

[Section to be written.]

Exercises

1. Given three anonymized stage summaries, diagnose each as skew, under-partitioned, or GC-bound, and name the evidence.

Chapter 16

Streaming into the Lakehouse

Structured Streaming is the same query engine over an unbounded table, processing one micro-batch at a time and ending each batch with a single Iceberg commit. The whole model from the earlier parts still applies — the shuffle, the executor budget, the commit — but with two new pressures: state that must be kept between batches, and a commit cadence that produces small files faster than a batch job. This chapter covers the streaming model and what it does to an Iceberg table.

16.1 The micro-batch model and the unbounded table

[Section to be written.]

16.2 Sources and sinks

[Section to be written.]

16.3 Event time and watermarking

[Section to be written.]

16.4 Stateful operations and the state store

[Section to be written.]

16.5 Output modes and triggers

[Section to be written.]

16.6 Exactly-once and the commit

[Section to be written.]

16.7 Writing to Iceberg: cadence, small files, maintenance

[Section to be written.]

16.8 Summary

[Section to be written.]

Exercises

1. Given a trigger interval and an input rate, estimate files committed per hour and the compaction cadence needed to keep the table healthy.

Part V

Internals

Chapter 17

Tungsten and Code Generation

Below the plan, a stage does not interpret operators row by row. Tungsten stores rows as packed off-heap bytes, and whole-stage code generation collapses an operator sub-tree into a single generated Java method that the JIT compiles to tight machine code. This chapter opens that layer: the binary row format, off-heap memory management, cache-aware computation, and the generated code you can actually read with `EXPLAIN CODEGEN`.

17.1 The Tungsten binary row format

[Section to be written.]

17.2 Off-heap memory management

[Section to be written.]

17.3 Cache-aware computation

[Section to be written.]

17.4 Whole-stage code generation internals

[Section to be written.]

17.5 Expression code generation

[Section to be written.]

17.6 Columnar processing and vectorization

[Section to be written.]

17.7 Summary

[Section to be written.]

Exercises

1. Run `EXPLAIN CODEGEN` on a filter-and-project query. Identify which operators were fused into one generated method and which were not.

Chapter 18

The Shuffle Framework and the Scheduler

Chapter 4 treated the shuffle as a wall. This chapter builds the wall: the shuffle manager and its writers, the map output tracker that tells reduce tasks where to fetch from, the DAG scheduler that cuts a plan into stages, the task scheduler that places tasks on executors, and the block transfer service that moves the bytes. It is the machinery behind every stage boundary and every straggler.

18.1 The shuffle manager and shuffle writers

[Section to be written.]

18.2 The map output tracker

[Section to be written.]

18.3 The DAG scheduler and the task scheduler

[Section to be written.]

18.4 RPC and block transfer

[Section to be written.]

18.5 Locality and speculation

[Section to be written.]

18.6 Summary

[Section to be written.]

Exercises

1. Trace one wide query from `collect()` to task completion, naming each scheduler component it passes through.

Chapter 19

DataSource V2 and Extensions

Iceberg is not built into Spark. It plugs in through the DataSource V2 API, and the planner negotiates with it: which filters can you apply, which columns do you need, can you do this aggregate yourself? Understanding that negotiation explains why some predicates prune and others do not. This chapter covers the DSv2 read and write API, the pushdown negotiation, how Iceberg reports splits and statistics, and how to add your own Catalyst rules and SQL extensions.

19.1 The DataSource V2 read and write API

[Section to be written.]

19.2 Pushdown negotiation

[Section to be written.]

19.3 How Iceberg reports splits and statistics

[Section to be written.]

19.4 Custom Catalyst rules and SQL extensions

[Section to be written.]

19.5 Plugins

[Section to be written.]

19.6 Summary

[Section to be written.]

Exercises

1. Write a query whose filter is pushed into Iceberg and one whose filter is not. Explain the difference from the DSv2 pushdown rules.

Appendix A

The Reference Environment and the Running Example

[Section to be written.]

Appendix B

Configuration Reference

[Section to be written.]

Appendix C

Reading Query Plans

A reference for the physical-plan operators the book refers to. Chapter 2 explains the compilation pipeline that produces these plans and how to read one quickly; this appendix is the catalogue you come back to. Everything here is as of the reference environment (Appendix A).

C.1 The six EXPLAIN modes

Form	Output
EXPLAIN	physical plan only
EXPLAIN EXTENDED	parsed, analyzed, optimized logical, and physical plans
EXPLAIN FORMATTED	physical plan as a tree + a numbered list expanding every node
EXPLAIN COST	optimized logical plan with the planner's row-count / size estimates
EXPLAIN CODEGEN	physical plan + the generated Java for each whole-stage stage
df.explain(mode=)	same five, as "simple", "extended", "formatted", "cost", "codegen"

C.2 Reading a physical plan

Read **bottom-up**: the leaves are scans, the root is the last operator to run. The tree connectors are +- for the child in line and :- plus a : spine for the first child of a binary operator (a join). A * prefix, written *(n), marks an operator fused into whole-stage-generated Java stage n (Section C.9).

C.3 Scan operators

Where data enters the plan.

Operator	Source
BatchScan	a DataSource V2 table — Iceberg on the reference stack
FileScan parquet / orc / csv	a file-based table through the Hive catalog (DataSource V1)
InMemoryTableScan	a cached DataFrame or CACHE TABLE
DataSourceV2ScanRelation	the logical-plan form of a V2 scan

```
BatchScan trips
  PartitionFilters: [event_ts >= 2026-03-01, event_ts <
    2026-03-08]
  PushedFilters:    [IsNotNull(route_id)]
  DataFilters:      [IsNotNull(route_id)]
  ReadSchema:       struct<route_id,arrival_delay_s,event_ts>
  Batched:          true
```

Listing C.1: A scan node’s fields

PartitionFilters filters resolved against partition values, so whole partitions (directories on object storage) are skipped without being opened. Empty on a partitioned table means the filter did not match the partition transform (Chapter 7).

PushedFilters filters handed to the file reader, which skips row groups using column min/max statistics in the footer and any bloom filters, without decompressing them.

DataFilters every filter on the scan, including the residue evaluated row-by-row after reading. DataFilters minus PushedFilters is what could not be pushed.

ReadSchema the columns actually read. Wider than the query needs means a SELECT * defeated column pruning.

Batched: true vectorized columnar reads are enabled (spark.sql.parquet.enableVectorizedReader); false falls back to row-at-a-time decoding.

C.4 Exchange operators

Every Exchange is a shuffle and a stage boundary: serialize, write to local disk, transfer over the network, re-read on the destination. Counting them is the fastest estimate of a query's cost.

Partitioning	In the plan	Created by
Hash	hashpartitioning(col, n)	joins, GROUP BY, repartition(n, col)
Range	rangepartitioning(col ASC, n)	global ORDER BY, repartitionByRange
Round-robin	roundrobinpartitioning(n)	repartition(n) with no column
Single	SinglePartition	global aggregate with no group key, coalesce(1)

```
Exchange hashpartitioning(route_id, 200), ENSURE_REQUIREMENTS,
[plan_id=42]
```

Listing C.2: A shuffle exchange

ENSURE_REQUIREMENTS means the planner's EnsureRequirements rule inserted this exchange because the operator above it needs its input distributed that way. ReuseExchange in a plan means two identical exchanges were deduplicated — one shuffle feeds both consumers.

BroadcastExchange is not a shuffle. It collects one small side to the driver and ships it whole to every executor; it appears only below the build side of a BroadcastHashJoin or BroadcastNestedLoopJoin.

C.5 Sort operators

```
Sort [route_id ASC NULLS FIRST], false, 0
```

The boolean is true for a global sort (a single ordered output across all partitions, preceded by a rangepartitioning exchange) and false for a local per-partition sort. A SortMergeJoin always shows two local sorts, one per input, between the exchanges and the join. Sorts need a sort buffer in execution memory and spill when it is exhausted (Chapter 3).

C.6 Join operators

Chapter 5 covers how the planner chooses; this is how to recognize the choice in a plan.

BroadcastHashJoin. BroadcastHashJoin [...], BuildRight (or BuildLeft) with a BroadcastExchange below the build side and *no* exchange on the probe (large) side. The cheapest join: no shuffle of the big table.

SortMergeJoin. SortMergeJoin [...], Inner with two child branches, each a Sort over an Exchange hashpartitioning. The default for two large inputs, and the most expensive pattern — two full shuffles plus two sorts.

ShuffledHashJoin. ShuffledHashJoin [...], BuildRight with two Exchange nodes but *no* sorts: one side is shuffled and built into a hash table in memory. Cheaper than sort-merge when the build side fits and the sort is the bottleneck; enabled by `spark.sql.join.preferSortMergeJoin=false` or a SHUFFLE_HASH hint.

BroadcastNestedLoopJoin. A non-equi join condition (`price > min_price`). $O(N \times M)$. Fine when one side is tiny; a warning sign on two large tables — usually a missing or non-equi join key.

CartesianProduct. Every row of one side with every row of the other. Unless it is a deliberate cross join, the ON clause is missing.

C.7 Aggregation operators

Aggregation is two-phase: a partial aggregate on each partition (map-side combine), a shuffle, then a final aggregate.

```
* (3) HashAggregate(keys=[route_id], functions=[avg(on_time)])
+- Exchange hashpartitioning(route_id, 200),
   ENSURE_REQUIREMENTS
   +- * (2) HashAggregate(keys=[route_id], functions=[
      partial_avg(on_time)])
   +- ...
```

Listing C.3: Two-phase aggregation

Operator	Chosen when	Memory
HashAggregate	all aggregate buffers are fixed-width (numeric, date, timestamp)	off-heap hash map; falls back to sort-based under memory pressure
ObjectHashAggregate	a buffer holds a variable-width or complex type (collect_list, collect_set, some UDAFs)	on-heap objects; falls back to sort-based past <code>spark.sql.objectHashAggregate.sortBased fallbackThreshold</code> (128) distinct keys
SortAggregate	hash aggregation is not feasible	needs input pre-sorted by the group keys; handles any size, slower

C.8 Filter and Project

Filter applies a predicate; Project selects columns and evaluates expressions. Both are usually fused into a neighbouring scan or join inside a `*(n)` block. A standalone Filter directly above a scan is worth a look: if its predicate is not also in the scan's `PushedFilters`, the pushdown failed and the predicate is being evaluated row-by-row after reading (Section C.11).

C.9 Whole-stage codegen markers

```
* (2) HashAggregate(keys=[city], functions=[sum(revenue)])
+- Exchange hashpartitioning(city, 200)
   +- * (1) HashAggregate(keys=[city], functions=[partial_sum(
       revenue)])
       +- * (1) Filter isnotnull(city)
           +- * (1) ColumnarToRow
               +- BatchScan sales [city, revenue]
```

`*(1)` is one generated Java method: scan, columnar-to-row, filter, and partial aggregate run without leaving it. The Exchange breaks the chain — you cannot fuse across a network boundary — so the final aggregate is `*(2)`, a second generated method on the post-shuffle data. The number is a sequential id, not an order or a priority. Operators with no star ran through the interpreter; Section 2.5 lists why a stage loses codegen.

C.10 AQE nodes in a final plan

When Adaptive Query Execution is on (Chapter 6), the plan the SQL tab shows after the query finishes is rooted at `AdaptiveSparkPlan isFinalPlan=true`. `isFinalPlan=false` is the compile-time plan from `explain()` before an action — it may change substantially at runtime.

`AQEShuffleRead coalesced` AQE merged small post-shuffle partitions after seeing their real sizes.

AQEShuffleRead (skew=true) AQE split one or more oversized partitions.

BroadcastQueryStage a SortMergeJoin was converted to a broadcast at runtime because the shuffled side turned out small.

SortMergeJoin [...], isSkew=true AQE detected and handled skew in this join.

C.11 Predicate-pushdown killers

Predicates that cannot be pushed to Parquet, ORC, or Iceberg, so they run row-by-row after the read (Chapter 7):

- a UDF: `WHERE my_udf(col) = v`
- a function or complex expression on the column: `WHERE length(city) > 5`,
`WHERE a + b > 100`
- a CAST on the column side: `WHERE CAST(s AS INT) > 100`
- a non-deterministic function: `WHERE rand() > 0.5`
- a leading-wildcard LIKE: `WHERE name LIKE '%smith'` (a trailing wildcard, `'smith%'`, does push)
- an OR with any non-pushable branch — the whole OR fails

Verification. Run `EXPLAIN FORMATTED`, find the scan, and check that each partition-column filter is in `PartitionFilters` and each data-column filter is in `PushedFilters`. If one is missing, it hit a killer above.

Appendix D

Glossary

[Section to be written.]

Abbreviations

Abbreviation	Full form
AQE	Adaptive Query Execution
CBO	Cost-Based Optimization
CoW / MoR	Copy-on-Write / Merge-on-Read
DAG	Directed Acyclic Graph
DPP	Dynamic Partition Pruning
DSv2	DataSource V2 API
GC	Garbage Collection
GCS	Google Cloud Storage
GKE	Google Kubernetes Engine
HMS	Hive Metastore
JIT	Just-In-Time (compilation)
JVM	Java Virtual Machine
OLAP	Online Analytical Processing
OLTP	Online Transaction Processing
RDD	Resilient Distributed Dataset
RPC	Remote Procedure Call
SMJ	Sort-Merge Join
SQL	Structured Query Language
UDF	User-Defined Function

Technologies

Technology	What it is
Apache Spark	a distributed compute engine
Catalyst	Spark SQL's extensible query optimizer
Tungsten	Spark's binary in-memory execution back end
Apache Iceberg	an open table format for analytic datasets on object storage
Apache Parquet	a columnar file format
Hive Metastore	a catalog service mapping table names to metadata
Apache Kyuubi	a multi-tenant SQL gateway in front of Spark
Google Cloud Storage	an S3-compatible object storage system
GKE	Google's managed Kubernetes; the cluster manager here
async-profiler	a low-overhead JVM sampling profiler

Appendix E

Further Reading

[Section to be written.]

References

- [1] Apache Iceberg. *Apache Iceberg Documentation*. <https://iceberg.apache.org/docs/latest/>. 2024.
- [2] Apache Iceberg. *Apache Iceberg Table Spec*. <https://iceberg.apache.org/spec/>. Format version 2. 2024.
- [3] Apache Kyuubi. *Apache Kyuubi Documentation*. <https://kyuubi.apache.org/docs/latest/>. 2024.
- [4] Apache Spark. *Spark SQL, DataFrames and Datasets Guide (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/sql-programming-guide.html>. 2024.
- [5] Apache Spark. *Tuning Spark (Spark 3.5.2)*. <https://spark.apache.org/docs/3.5.2/tuning.html>. 2024.
- [6] Michael Armbrust et al. “Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics”. In: *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*. 2021.
- [7] Michael Armbrust et al. “Spark SQL: Relational Data Processing in Spark”. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 1383–1394.
- [8] Michael Armbrust et al. “Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark”. In: *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. ACM, 2018, pp. 601–613.
- [9] Bill Chambers and Matei Zaharia. *Spark: The Definitive Guide*. O’Reilly Media, 2018.

- [10] Jules S. Damji et al. *Learning Spark: Lightning-Fast Data Analytics*. 2nd ed. O'Reilly Media, 2020.
- [11] Holden Karau, Adi Polak, and Rachel Warren. *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark*. 2nd ed. O'Reilly Media, 2026.
- [12] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [13] Tomer Shiran, Jason Hughes, and Alex Merced. *Apache Iceberg: The Definitive Guide*. O'Reilly Media, 2024.
- [14] Matei Zaharia et al. "Apache Spark: A Unified Engine for Big Data Processing". In: *Communications of the ACM* 59.11 (2016), pp. 56–65.
- [15] Matei Zaharia et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2012, pp. 15–28.